

UNIT 3

1. What is normalization ? What is the need of normalized database ? Explain 1NF, 2NF, 3NF and BCNF with example.

Normalization is a systematic approach to organizing data in a database to reduce redundancy and improve data integrity. This process involves dividing a database into two or more tables and defining relationships between the tables. The main objective is to ensure that each table contains data related to a specific topic, which helps in maintaining the data efficiently.

Need for a Normalized Database

1. It eliminates redundant data.
2. It reduces chances of data error.
3. The normalization is important because it allows database to take up less disk space.
4. It also help in increasing the performance.
5. It improves the data integrity and consistency.

2. Explain with example requirements of first normal form.

The **First Normal Form (1NF)** is the most basic level of database normalization. A relation is in 1NF if it satisfies the following requirements:

Requirements of 1NF:

1. Atomic Values (No Multivalued Attributes):

- Each cell in the table must contain a single, indivisible value. There should be no sets, lists, or arrays within a cell.

2. Unique Rows:

- Each row in the table must be unique, meaning it should be identifiable by a primary key.

3. No Repeating Groups:

- The table should not have repeating groups or arrays of values. Instead, each piece of information should be stored in its own column.
- Each column must contain values of a single type.

Example:

Table Not in 1NF (Before Normalization):

StudentID	Name	Subjects	Contact Numbers
1	Alice	Math, Physics	12345, 67890
2	Bob	Chemistry, Biology	54321, 98765, 11223

- **Issues:**

1. **Multivalued Attributes:** Columns like Subjects and Contact Numbers contain multiple values.
2. **Repeating Groups:** There are no unique rows; data is grouped in a single cell.

Table in 1NF (After Normalization):

StudentID	Name	Subject	Contact Number
1	Alice	Math	12345
1	Alice	Physics	67890
2	Bob	Chemistry	54321
2	Bob	Biology	98765
2	Bob	Biology	11223

Fixes:

1. Each cell contains atomic values (single values).
2. Repeating groups have been eliminated by creating multiple rows for each subject or contact number.
3. Each row is uniquely identifiable by a combination of StudentID, Subject, and Contact Number.

3. Explain with example requirements of second normal form.

The **Second Normal Form (2NF)** builds on the First Normal Form (1NF) and is achieved by eliminating partial dependencies. A relation is in 2NF if:

Requirements of 2NF:

1. **The Relation Must Be in 1NF:**

- The table should first satisfy all the requirements of the First Normal Form.

2. **No Partial Dependency:**

- A **partial dependency** exists when a non-prime attribute (an attribute not part of any candidate key) depends on part of a composite primary key rather than the whole key.

- To eliminate partial dependencies, non-prime attributes must depend on the **entire** primary key (if a composite key exists).

Example:

Table Not in 2NF (Before Normalization):

StudentID	CourseID	CourseName	InstructorName
1	101	Math	Dr. Smith
1	102	Physics	Dr. Brown
2	101	Math	Dr. Smith
2	103	Chemistry	Dr. Johnson

- **Primary Key:** (StudentID, CourseID) (Composite Key).
- **Partial Dependency:**
 - CourseName and InstructorName depend only on CourseID, not on the entire composite key (StudentID, CourseID).
 - This violates 2NF.

Table in 2NF (After Normalization):

Decomposed into Two Tables:

1. Student-Course Table (Eliminating Partial Dependency):

StudentID	CourseID
1	101
1	102
2	101
2	103

2. Course Table (Attributes Depending Only on CourseID):

CourseID	CourseName	InstructorName
101	Math	Dr. Smith
102	Physics	Dr. Brown
103	Chemistry	Dr. Johnson

- **Fixes:**
 1. All non-prime attributes now depend only on the primary key of their respective tables.
 2. CourseName and InstructorName are moved to a separate table where CourseID is the primary key, eliminating partial dependency.

4. Explain 3NF and BCNF and give its example. Also enlist their differences.

The **Third Normal Form (3NF)** aims to eliminate **transitive dependencies** while building on the rules of the Second Normal Form (2NF). A relation is in 3NF if:

Requirements of 3NF:

1. The Relation Must Be in 2NF:

- The table should already satisfy all the requirements of the Second Normal Form.

2. No Transitive Dependency:

- A **transitive dependency** occurs when a non-prime attribute (an attribute not part of any candidate key) depends on another non-prime attribute, which in turn depends on the primary key.
- To eliminate transitive dependencies, non-prime attributes must depend **only** on the primary key.

3. Every Non-Prime Attribute Must Be Directly Dependent on the Primary Key:

- Non-prime attributes cannot depend on other non-prime attributes.

Example:

Table Not in 3NF (Before Normalization):

StudentID	StudentName	CourseID	CourseName	InstructorName
1	Alice	101	Math	Dr. Smith
1	Alice	102	Physics	Dr. Brown
2	Bob	101	Math	Dr. Smith
2	Bob	103	Chemistry	Dr. Johnson

- **Primary Key:** (StudentID, CourseID).
- **Transitive Dependency:**
 - CourseName and InstructorName depend on CourseID, and CourseID depends on the composite key (StudentID, CourseID).
 - Hence, CourseName and InstructorName are **transitively dependent** on (StudentID, CourseID) via CourseID.

Table in 3NF (After Normalization):

Decomposed into Three Tables:

1. Student Table (Eliminating Transitive Dependency):

StudentID	StudentName
1	Alice
2	Bob

2. Student-Course Table:

StudentID	CourseID
1	101
1	102
2	101
2	103

3. Course Table:

CourseID	CourseName	InstructorName
101	Math	Dr. Smith
102	Physics	Dr. Brown
103	Chemistry	Dr. Johnson

- **Fixes:**

1. CourseName and InstructorName are moved to a separate table where CourseID is the primary key, eliminating the transitive dependency.
2. Now, all non-prime attributes in each table depend **only** on the primary key.

Boyce-Codd Normal Form (BCNF)

A 3NF table which does not have multiple overlapping candidate keys is said to be in BCNF.

A relation is in BCNF if:

Requirements of BCNF:

1. The Relation Must Be in 3NF:

- The table should first satisfy all the requirements of the Third Normal Form.

2. No Overlapping Candidate Keys:

For each functional dependency ($X \rightarrow Y$), X should be a super key. In simple words if Y is a prime attribute then X can not be non prime attribute.

Example:

Enrollment table

sid	Course	Teacher
1	C	Ankita
1	Java	Poonam
2	C	Ankita
3	C++	Supriya
4	C	Archana

From above table following observations can be made :

- One student can enroll for multiple courses. For example student with sid = 1 can enroll for C as well as Java.
- For each course, a teacher is assigned to the student.
- There can be multiple teachers teaching one course for example course C can be taught by both the teachers namely - Ankita and Archana.
- The candidate key for above table can be (sid, course), because using these two columns we can find,
- The above table holds following dependencies $(sid, course) \rightarrow Teacher$ or $Teacher \rightarrow course$
- The above table is not in BCNF because of the dependency $teacher \rightarrow course$. Note that the teacher is not a superkey or in other words, teacher is a non prime attribute and course is a prime attribute and non-prime attribute derives the prime attribute.
- To convert the above table to BCNF we must decompose above table into student and course tables.

Student

sid	Teacher
1	Ankita
1	Poonam
2	Ankita
3	Supriya
4	Archana

Course

Teacher	Course
Ankita	C
Poonam	Java
Ankita	C
Supriya	C++
Archana	C

Sr. No.	3NF	BCNF
1.	3NF stands for Third Normal Form.	BCNF stands for Boyce Codd Normal Form.
2.	The table is in 3NF if it is in 2NF and for each functional dependency $X \rightarrow Y$ at least following condition hold: (i) X is a superkey, (ii) Y is prime attribute of table.	The table is in BCNF if it is in 3rd normal form and for each relation $X \rightarrow Y$ X should be super key.
3.	3NF can be obtained without sacrificing all dependencies.	Dependencies may not be preserved in BCNF.
4.	Lossless decomposition can be achieved in 3NF.	Lossless decomposition is hard to obtain in BCNF.
5.	3NF can be achieved without losing any information from the old table.	For obtaining BCNF we may lose some information from old table.

5. Elaborate the significance of codd's rule. Explain 12 rules proposed by codd's.

Rule 0: Foundation Rule

This rule states for a database to be relational, it must use its relational capabilities to manage the database. This means it should support relational integrity, independence, and operations entirely through relational principles.

Rule 1: Information Rule

All data in an RDBMS should be stored in table format, with rows and columns. Data must be represented as values in tables, allowing a uniform, logical view of all data. Example:

Imagine a "Students" table where each row represents a student, and columns represent attributes like student_id, name, and age.

Rule 2: Guaranteed Access Rule

Each piece of data (value) must be accessible by specifying its table name, column name, and primary key. This guarantees that any data value can be accessed without ambiguity. *Example:* To access a student's name, you'd use the table name (Students), primary key (student_id), and column name (name).

Rule 3: Systematic Treatment of Null Values

An RDBMS must handle null values consistently, representing missing or inapplicable information, and not just using zero or empty strings. Nulls should be treated systematically and without compromising logical operations or integrity. *Example:* If a student doesn't have a middle name, the middle_name column would store a null value to represent missing data.

Rule 4: Dynamic Online Catalog Based on the Relational Model

Database dictionary which is called as catalog-is the structure description of the complete Database and it must be stored online. This Catalog must be governed by same rules as rest of the database. The same query language should be used on catalog as used to query database. *Example:* Metadata about tables and columns can be queried using SQL statements.

Rule 5: Comprehensive Data Sublanguage Rule

Database should not be directly accessible. It should always be accessible by using some strong query language. This rule illustrates that the system should support at least one relational language. The language –

- a) has a linear syntax
- b) can be used both interactively and within application programs,
- c) supports data definition operations (including view definitions), data manipulation operations (update as well as retrieval), security and integrity constraints and transaction management operations (begin, commit, and rollback).

Example: SQL (Structured Query Language) is used to perform operations like SELECT, INSERT, UPDATE, and DELETE.

Rule 6: View Updating Rule

All views in the database should be theoretically updatable. If a view represents a specific part of the database, changes to that view should propagate back to the underlying data tables, ensuring consistency. *Example:* A view showing only students over 18 should be updatable if it doesn't conflict with other constraints.

Rule 7: High-Level Insert, Update, and Delete

An RDBMS must support high-level data manipulation commands, allowing rows to be inserted, updated, or deleted as sets (not just one at a time). This set-based manipulation ensures operations on multiple rows can be performed simultaneously. *Example:* Using SQL, you can insert a new student record or update existing records without writing low-level code.

Rule 8: Physical Data Independence

Physical storage should not matter the system. Whenever any changes are made in either storage representations or access methods then it should not affect the application. *Example:* Moving data from one storage device to another shouldn't change how you query the data.

Rule 9: Logical Data Independence

If any changes are made in table structure then the logical view of the user should not get affected. For Rule example – if a table is split into two tables internally, the view of the table to the user should be an entire table and not the split tables. *Example:* Adding a new column to the "Students" table shouldn't break existing applications that query the table.

Rule 10: Integrity Independence

The Integrity constraints must be defined by the RDBMS stored in the system and it should not be enforced by the external application programs. *Example:* A constraint to ensure that student ages are between 5 and 100 should be defined in the database schema, not in application code.

Rule 11: Distribution Independence

An RDBMS must have distribution independence. That means, even if database is scattered geographically, user should get a feel as if it is stored in one piece at one location. *Example:* Whether student data is stored on a single server or distributed across multiple servers should not affect how you query it.

Rule 12: Non subversion Rule

If low-level language is allowed to access the system, then that low-level language must not be able to subvert or bypass the integrity rules which are expressed in a higher-level language. *Example:* Direct access to the database files should not allow inserting a student record with an invalid student_id.

6. What are the desirable properties of decomposition? Explain it with example.

Decomposition is the process of breaking down a complex database schema into smaller, simpler, and more manageable parts (or relations) while preserving the integrity and correctness of the data. The primary goal is to reduce redundancy, avoid anomalies, and improve database efficiency.

Desirable Properties of Decomposition

- 1. Lossless-Join Property**
- 2. Dependency Preservation Property**

1. Lossless-Join Property

Definition: A decomposition is lossless if, when the decomposed tables are joined, they exactly recreate the original table without any loss of information.

This is achieved by maintaining a common attribute (a key) that allows the original relation to be reconstructed.

The lossless join can be defined using following three conditions :

i) Union of attributes of R1 and R2 must be equal to attribute of R. Each attribute of R must be either in R1 or in R2.

$$\text{Att}(R1) \cup \text{Att}(R2) = \text{Att}(R)$$

ii) Intersection of attributes of R1 and R2 must not be NULL.

$$\text{Att}(R1) \cap \text{Att}(R2) \neq \Phi$$

iii) Common attribute must be a key for at least one relation (R1 or R2)

$$\text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R1)$$

or

$$\text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R2)$$

Example:

Suppose we have a relation R with attributes R(A,B,C)R(A, B, C)R(A,B,C), and it is decomposed into two relations:

- R1(A,B)R1(A, B)R1(A,B)
- R2(B,C)R2(B, C)R2(B,C)

If BBB is a **key** in R1R1R1 or R2R2R2, then the decomposition is lossless because joining R1R1R1 and R2R2R2 on BBB will reconstruct $R(A,B,C)R(A, B, C)R(A,B,C)$.

Consider a table R with attributes {student_id, student_name, course_id, course_name}.

student_id	student_name	course_id	course_name
1	John Doe	101	Math
2	Jane Smith	102	Science

Decompose R into two tables: **Students Table**:

student_id	student_name
1	John Doe
2	Jane Smith

Enrollments Table:

student_id	course_id	course_name
1	101	Math
2	102	Science

Lossless-Join: Joining "Students" and "Enrollments" on student_id perfectly reconstructs the original table.

2. Dependency Preservation Property

Definition: A decomposition is dependency-preserving if all functional dependencies are preserved in the decomposed tables, enabling constraint enforcement without needing to join tables.

If we decompose a relation R into relations R1 and R2, All dependencies of R either must be a part of R1 or R2 or must be derivable from a combination of [functional dependency](#) of R1 and R2.

Example: Given functional dependencies in R:

- $\text{student_id} \rightarrow \text{student_name}$
- $\text{course_id} \rightarrow \text{course_name}$

When decomposed: **Students Table**:

student_id student_name

1 John Doe
2 Jane Smith

Enrollments Table:

student_id course_id course_name

1	101	Math
2	102	Science

Both functional dependencies are preserved:

- $\text{student_id} \rightarrow \text{student_name}$ is preserved in the "Students" table.
- $\text{course_id} \rightarrow \text{course_name}$ is preserved in the "Enrollments" table.

7. Explain partial and full dependencies and transitive dependencies with example.

Full Functional Dependency

Definition: A full functional dependency occurs when a non-key attribute is functionally dependent on the entire primary key, not just a part of it.

Example: Consider a table "Enrollments" with the following structure:

student_id	course_id	grade
1	101	A
1	102	B
2	101	B
2	103	A

In this example, the primary key is the combination of `student_id` and `course_id`. The attribute `grade` is fully functionally dependent on both `student_id` and `course_id` together. You cannot determine the `grade` by knowing just one of these columns; you need the entire primary key.

Partial Dependency

Definition: A partial dependency occurs when a non-key attribute is functionally dependent on a part of the primary key, rather than the whole primary key.

Example: Consider a table "StudentCourses" with the following structure:

student_id	course_id	student_name
1	101	John Doe
1	102	John Doe
2	101	Jane Smith
2	103	Jane Smith

Here, student_id and course_id together form the primary key. However, student_name depends only on student_id and not on the entire primary key. This is a partial dependency because student_name is dependent on part of the composite primary key (student_id), not on both student_id and course_id.

Transitive Dependencies

Definition: A transitive dependency in a relational database occurs when a non-prime attribute depends on another non-prime attribute through a third attribute. In other words, a non-prime attribute is indirectly dependent on the primary key. This can lead to redundancy and anomalies, such as update, insert, and delete anomalies.

Explanation: In the context of functional dependencies, if we have $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ is a transitive dependency. Here, AA is a primary key or a composite key, BB is a non-prime attribute, and CC is also a non-prime attribute.

8. What is the impact of insert, update & delete anomaly on overall design of database? How normalization is used to remove these anomalies?

Anomalies in the Relational Model

Insert, update, and delete anomalies are problems that can occur in poorly designed databases where redundancy and dependency are not properly managed. These anomalies can lead to data inconsistency and affect the reliability of the database. Let's explore each type of anomaly and how normalization helps to eliminate them:

Insert Anomaly

Definition: An insert anomaly occurs when certain attributes cannot be inserted into the database without the presence of other attributes.

Impact:

- It can prevent the addition of new records.
- Leads to incomplete or missing data.

Example: Consider a table Students_Courses that records student enrollments:

StudentID	StudentName	CourseID	CourseName
101	Alice	C101	Math
102	Bob	C102	Science

If we want to add a new course to the database but no student has enrolled in it yet, we cannot insert the course information without a corresponding StudentID and StudentName.

Normalization Solution: Normalization helps by decomposing the table into two tables:

1. **Students:**

StudentID	StudentName
101	Alice
102	Bob

2. **Courses:**

CourseID	CourseName
C101	Math
C102	Science

3. **Enrollments:**

StudentID	CourseID
101	C101
102	C102

This way, new courses can be added to the Courses table without requiring student information.

Update Anomaly

Definition: An update anomaly occurs when a single update operation must be carried out on multiple instances of the same data, leading to data inconsistency.

Impact:

- Inconsistent data after updates.
- Increased risk of errors when updating data.

Example: Consider a table Employees that includes department information:

EmployeeID	EmployeeName	DepartmentID	DepartmentName
1	John	D1	HR

2	Jane	D1	HR
3	Mike	D2	IT

If the department name "HR" changes to "Human Resources," we must update it in multiple rows. If we miss any row, the data becomes inconsistent.

Normalization Solution: Normalization helps by decomposing the table into two tables:

1. **Employees:**

EmployeeID	EmployeeName	DepartmentID
1	John	D1
2	Jane	D1
3	Mike	D2

2. **Departments:**

DepartmentID	DepartmentName
D1	Human Resources
D2	IT

Now, updating the department name requires only a single update in the Departments table.

Delete Anomaly

Definition: A delete anomaly occurs when the deletion of data representing one fact necessitates the deletion of data representing another fact.

Impact:

- Unintended loss of important data.
- Data dependency issues.

Example: Consider a table Projects that includes employee assignment:

ProjectID	ProjectName	EmployeeID	EmployeeName
1001	Alpha	1	John
1002	Beta	2	Jane

If the project "Alpha" is completed and we delete the record for ProjectID = 1001, we also lose the employee information for John.

Normalization Solution: Normalization helps by decomposing the table into two tables:

1. **Projects:**

ProjectID	ProjectName
1001	Alpha
1002	Beta

2. **EmployeeAssignments:**

EmployeeID	ProjectID
1	1001
2	1002

This way, deleting a project does not remove the employee information.

9. What is functional dependency? Explain its use in database design. Consider the following schema. Student (RollNo, Branch_code, Marks_Obtained, Exam_Name, Total_Marks)

Identify the functional dependencies and check whether the given schema is in 3NF or not. If not justify and convert the schema into 3NF.

Functional Dependency

Definition: Functional dependency is a constraint between two sets of attributes in a relational database. A functional dependency, denoted as $A \rightarrow B$, indicates that for a given value of attribute A, there is exactly one value of attribute B. In other words, A functionally determines B.

Use in Database Design: Functional dependencies are crucial in database design for:

1. **Normalization:** Ensuring that the database schema is designed in such a way that it reduces redundancy and avoids update, insert, and delete anomalies.
2. **Data Integrity:** Maintaining the consistency and accuracy of data by enforcing constraints based on functional dependencies.
3. **Identifying Candidate Keys:** Determining the minimal set of attributes needed to uniquely identify a tuple in a relation.

Example of Functional Dependencies

Consider a schema Student (RollNo, Branch_code, Marks_Obtained, Exam_Name, Total_Marks):

Identifying Functional Dependencies

1. **RollNo** → **Branch_code**: A student's RollNo uniquely determines their Branch_code.
2. **RollNo, Exam_Name** → **Marks_Obtained**: A student's RollNo and Exam_Name together determine the Marks_Obtained.
3. **Exam_Name** → **Total_Marks**: An Exam_Name uniquely determines the Total_Marks.

Checking 3NF

To check if the given schema is in 3NF, we need to verify:

1. It is in 2NF.
2. There are no transitive dependencies.

Current Schema:

RollNo Branch_code Marks_Obtained Exam_Name Total_Marks

Analysis

- **Functional Dependencies:**
 - RollNo → Branch_code
 - RollNo, Exam_Name → Marks_Obtained
 - Exam_Name → Total_Marks

Issues:

- Transitive Dependency: RollNo determines Branch_code and Exam_Name determines Total_Marks. This means Branch_code and Total_Marks are dependent on non-key attributes (Exam_Name).

Converting to 3NF

Step 1: Decompose the table to eliminate transitive dependencies.

Students Table:

RollNo	Branch_code
1	CS
2	EE

Exams Table:

Exam_Name	Total_Marks
Math	100
Physics	100

Marks Table:

RollNo	Exam_Name	Marks_Obtained
1	Math	90
1	Physics	80
2	Math	85

Explanation

1. **Students Table:**

- RollNo determines Branch_code (no transitive dependencies).

2. **Exams Table:**

- Exam_Name determines Total_Marks (no transitive dependencies).

3. **Marks Table:**

- RollNo and Exam_Name together determine Marks_Obtained (no transitive dependencies).

By decomposing the original table into these three tables, we ensure that each non-key attribute depends only on the primary key, achieving 3NF and eliminating transitive dependencies.

10. What is decomposition? Consider the relation F (FN, PN, C, D) with the following Functional Dependencies:

FD1: FN, PN → C

FD2: C → D

FD3: D → F

If F is decomposed in to F1 (FN,PN,C) and F2 (C,D). check decomposition is lossless or lossy?

What is Decomposition?

Decomposition is the process of breaking down a relation into smaller relations to achieve a higher normal form, reduce redundancy, and avoid anomalies while preserving the original data integrity. Decomposition can be:

1. **Lossless (Non-Lossy):**

- The original relation can be perfectly reconstructed by joining the decomposed relations.
- This is crucial for ensuring no data is lost during decomposition.

2. **Lossy:**

- The original relation **cannot** be perfectly reconstructed from the decomposed relations.
- This results in a loss of information.

For a decomposition to be **lossless**, at least one of the following conditions must hold:

- The common attribute(s) between decomposed relations must form a **superkey** in at least one of the relations.

Analysis:

1. The attribute C is present in both F_1 and F_2 , so it acts as the **common attribute**.
2. From $FD2 (C \rightarrow D)$, we know that C determines D .
3. From the transitive closure of the FDs, we find that C can determine all attributes (C is a superkey for F).

Conclusion:

- The decomposition $F_1(FN, PN, C)$ and $F_2(C, D)$ is **lossless** because the common attribute C is a superkey for F .

11. What are relational integrity constraints. Explain with example Domain constraints, Referential-Integrity and enterprise constraints.

Relational Integrity Constraints

Relational Integrity Constraints are rules that ensure the accuracy, consistency, and validity of data in a relational database. They enforce data integrity by restricting the types of operations that can be performed on the data. The most common types of integrity constraints are:

1. **Domain Constraints**
2. **Referential Integrity Constraints**
3. **Enterprise Constraints**

1. Domain Constraints

Definition:

Domain constraints specify the permissible values for an attribute in a relation. Each attribute has a domain, or a set of allowable values, that it can take.

Purpose:

To ensure that data entered in a column is valid and consistent.

Example:

- Consider a table **Student**:

StudentID	StudentName	Age	Gender
1	Alice	20	F
2	Bob	22	M

- The Age column has a domain constraint: $18 \leq \text{Age} \leq 25$
- The Gender column has a domain constraint: $\{M, F\}$.

If someone tries to insert a record with Age = 15 or Gender = X, it will violate the domain constraint.

2. Referential Integrity Constraints

Definition:

Referential integrity ensures that a foreign key in one table corresponds to a valid primary key in another table. It enforces relationships between tables.

Purpose:

To maintain consistency between related tables.

Example:

- Consider two tables:

Student Table:

StudentID	StudentName	BranchID
1	Alice	101
2	Bob	102

- Branch** Table:

BranchID	BranchName
101	Computer Sci
102	Mechanical

- BranchID in the **Student** table is a foreign key referencing the BranchID in the **Branch** table.
- Referential integrity ensures that every value of BranchID in **Student** must exist in **Branch**.
- If someone tries to insert a record in **Student** with BranchID = 105 (which doesn't exist in **Branch**), the constraint is violated.

3. Enterprise Constraints

Definition:

Enterprise constraints are business-specific rules or conditions that must be satisfied by the data. These are not standard relational constraints but are defined based on business logic.

Purpose:

To enforce organizational rules and policies.

Example:

- Consider a **Loan** table:

LoanID	CustomerID	Amount	LoanType
1	1001	5000	PersonalLoan
2	1002	10000	HomeLoan

- **Enterprise Constraint:**
 - The **Amount** for a Personal Loan cannot exceed 10,000.
 - If someone tries to insert a Personal Loan with Amount = 15,000, it will violate the enterprise constraint.

12. Explain the concepts of referential integrity constraint and entity integrity constraint with example

i) Entity Integrity Constraint

- This rule states that "In the relations, the value of attribute of primary key can not be null".
- The NULL represents a value for an attribute that is currently unknown or is not applicable for this tuple. The Nulls are always to deal with incomplete or exceptional data.
- The primary key value helps in uniquely identifying every row in the table. Thus if the users of the database want to retrieve any row from the table or perform any action on

that table, they must know the value of the key for that row. Hence it is necessary that the primary key should not have the NULL value.

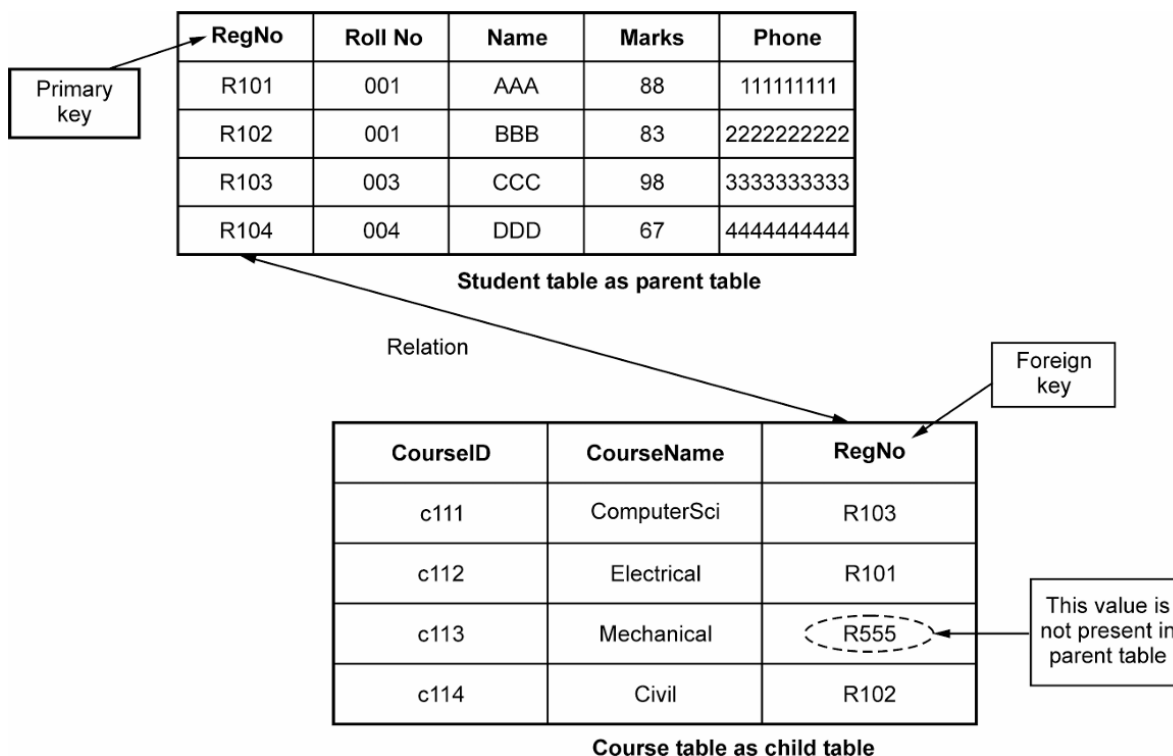
- For example -

Roll No	Name	Marks	Phone
001	AAA	88	111111111
001	BBB	83	222222222
003	CCC	98	333333333
	DDD	67	444444444

NULL is not allowed !!!

ii) Referential Integrity Constraint

- In relationships, data is linked between two or more tables.
- This is achieved by having the foreign key (in the associated table) reference a primary key value (in the primary - or parent - table). Because of this, we need to ensure that data on both sides of the relationship remain intact.
- The referential integrity rule states that “whenever a foreign key value is used it must reference a valid, existing primary key in the parent table”.
- For example - Consider two tables.



13. Explain different features of good relational database design.

Good relational database design ensures data integrity, performance, and scalability while minimizing redundancy and maintaining ease of maintenance. Here are the key features of a well-designed relational database:

1. Data Normalization

- **Objective:** Reduce redundancy and improve data integrity.
- **Details:** Breaking down tables to eliminate duplicate data by following normalization rules (1NF, 2NF, 3NF, etc.).
- **Benefit:** Avoids anomalies during insert, update, or delete operations.

2. Data Integrity

- **Objective:** Ensure the correctness and consistency of data.
- **Types of Integrity:**
 - **Entity Integrity:** Each table should have a unique primary key.
 - **Referential Integrity:** Foreign keys must correspond to valid primary keys in related tables.
 - **Domain Integrity:** Data should conform to a defined domain (e.g., data types, ranges, and constraints).

3. Clear and Logical Schema Design

- **Objective:** Organize data in a logical structure.
- **Details:**
 - Properly naming tables and columns to reflect their purpose.
 - Using appropriate data types for each attribute.
 - Defining relationships explicitly (one-to-one, one-to-many, or many-to-many).

4. Scalability

- **Objective:** Handle growth in data volume or user load.
- **Details:**
 - Indexing frequently queried columns to speed up data retrieval.
 - Avoiding excessive joins for better performance.

- Designing with future scaling in mind.

5. Efficient Query Design

- **Objective:** Support high-performance queries.
- **Details:**
 - Structuring queries and indexes for optimal performance.
 - Denormalizing specific parts of the database if needed for heavy read operations.

6. Security and Access Control

- **Objective:** Protect sensitive data.
- **Details:**
 - Implementing user roles and permissions.
 - Encrypting sensitive data.
 - Avoiding overexposure by limiting access to necessary data only.

7. Flexibility and Extensibility

- **Objective:** Accommodate future changes without major redesigns.
- **Details:**
 - Using surrogate keys for unique identification.
 - Designing tables that can evolve with new attributes or relationships.

8. Performance Optimization

- **Objective:** Ensure fast and efficient operations.
- **Details:**
 - Using indexes, partitioning, and caching.
 - Minimizing the use of complex joins and subqueries.
 - Monitoring and tuning performance with real-world workloads.

9. Data Consistency

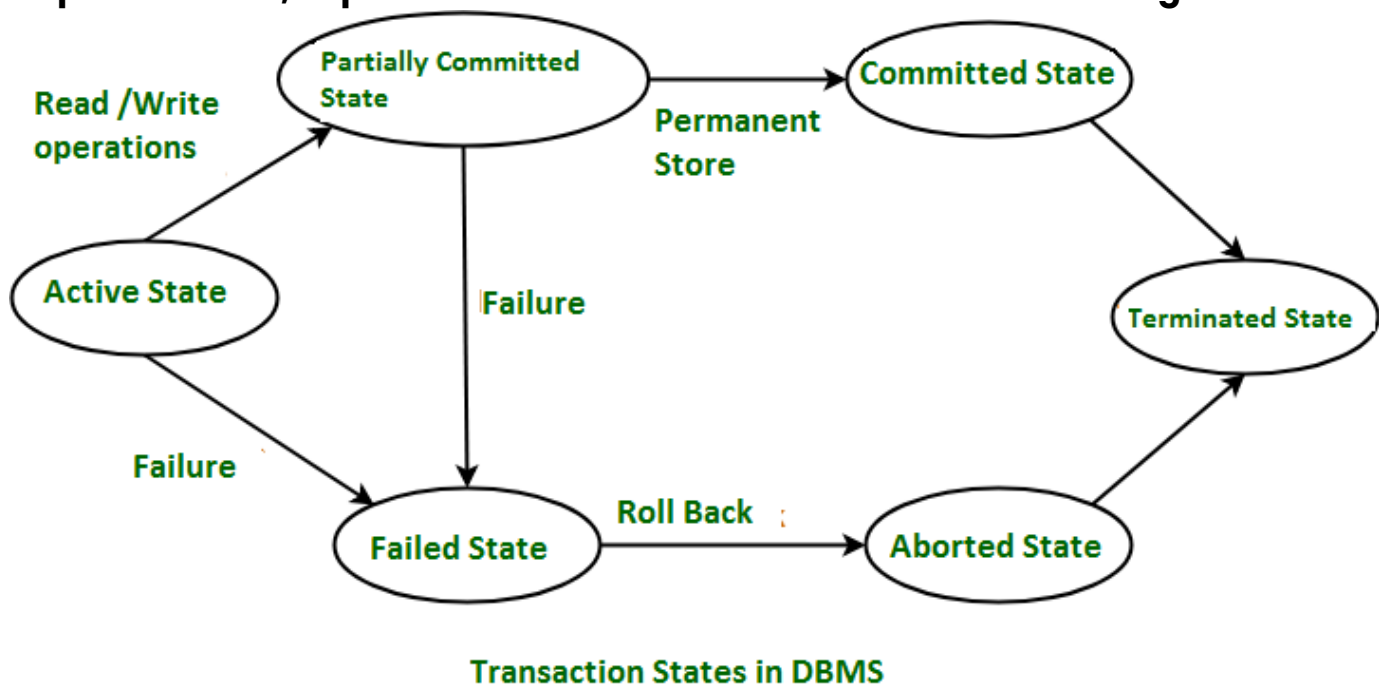
- **Objective:** Maintain uniform and valid data across the database.
- **Details:** Enforcing constraints like UNIQUE, NOT NULL, and CHECK.

10. Backup and Recovery

- **Objective:** Ensure data can be restored in case of failure.
- **Details:**
 - Regular backups and snapshots.
 - Designing recovery plans and strategies.

UNIT 4

1. Transaction during its execution should be in one of the different states at any point of time, explain the different states of transactions during its execution.



1. Active State

- **Definition:** The transaction is in the active state when it is being executed. It involves operations such as reading, writing, and updating the data.
- **Key Points:**
 - The transaction is initiated.
 - All operations are performed in this state.
 - The transaction can either complete successfully or encounter an error.

2. Partially Committed State

- **Definition:** Once a transaction has executed its final operation, it enters the partially committed state.
- **Key Points:**
 - All operations have been completed.
 - The system is now in the process of writing the changes to the database.
 - The transaction may still fail at this stage if there is a system crash.

3. Failed State

- **Definition:** The transaction moves to the failed state if any error occurs during its execution or if it is aborted.
- **Key Points:**
 - Errors could be due to logical errors, constraint violations, or system crashes.
 - The database must be rolled back to its previous consistent state.
 - All changes made by the transaction are undone.

4. Aborted State

- **Definition:** When a transaction cannot proceed or fails, it moves to the aborted state. The changes made by the transaction are rolled back.
- **Key Points:**
 - The transaction can be restarted or terminated based on the system's requirements.
 - A rollback operation ensures the database remains consistent.

5. Committed State

- **Definition:** After successfully executing all operations and ensuring the changes can be permanently saved, the transaction enters the committed state.
- **Key Points:**
 - All changes made by the transaction are permanently recorded in the database.
 - The transaction is considered successfully completed.
 - Once in this state, the transaction cannot be undone.

6. Terminated State

- **Definition:** A transaction reaches the terminated state after it has been committed or aborted.
- **Key Points:**
 - The transaction's lifecycle is complete.
 - The system can now remove the transaction from its active list.

2. Explain the concept of conflict serializability with suitable example. Since every conflict-serializable schedule is view serializable, why do we emphasize conflict serializability rather than view serializability?

While **view serializability** is a more general concept than **conflict serializability**, we emphasize **conflict serializability** because it is easier to understand, test, and implement in database systems. Let's break this down:

1. Definitions

- **Conflict serializability:** A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.
- **View serializability:** A schedule is view-serializable if it is equivalent to a serial schedule in terms of:
 1. **Initial Reads:** Each transaction reads the same initial value in both schedules.
 2. **Final Writes:** The final write to each data item must be the same in both schedules.
 3. **Read-Write Dependencies:** If a transaction reads a value written by another transaction, the same dependencies must hold.

2. Why Conflict Serializability is Preferred

a) Simpler Verification

- **Conflict serializability** can be verified using a **precedence graph** by checking for cycles. This is computationally efficient and practical.
- **View serializability**, on the other hand, is harder to verify because it requires checking read-write dependencies, initial reads, and final writes. The problem of testing view serializability is **NP-complete**, making it infeasible for real-time systems.

b) Stronger Condition

- Conflict serializability is a subset of view serializability. This means every conflict-serializable schedule is view-serializable, but not vice versa.
- By enforcing conflict serializability, we ensure view serializability while avoiding the computational overhead of checking the more general condition.

c) Practical Implementations

- Most concurrency control mechanisms (e.g., **locking protocols, timestamp ordering**) are designed to ensure conflict serializability. These mechanisms are efficient and widely used.
- Ensuring view serializability requires more complex algorithms, which are rarely implemented because of their cost and complexity.

d) Clear Conceptual Understanding

- Conflict serializability is more intuitive and easier to understand for database developers and administrators. It focuses on specific, identifiable conflicts between operations.

3. When View Serializability Might Be Needed

In rare cases, schedules may be view-serializable but not conflict-serializable. This happens when operations like blind writes (a write operation not dependent on a prior read) are involved. However, such schedules are uncommon in typical database transactions, so conflict serializability suffices for most practical scenarios.

3. State and explain the ACID properties.

1. Atomicity

- **Definition:** Atomicity ensures that a transaction is treated as a single, indivisible unit of work. Either all its operations are executed successfully, or none of them are applied.
- **Explanation:** If a transaction fails midway (e.g., due to a power failure), the database should revert to its previous state, leaving no partial updates. This guarantees that the database is never left in an inconsistent or partial state.
- **Example:** A bank transfer transaction involves debiting one account and crediting another. If one of these steps fails, neither operation should be applied.

2. Consistency

- **Definition:** Consistency ensures that a transaction transforms the database from one valid state to another, maintaining the integrity of the data.

- **Explanation:** Any transaction must satisfy all the defined rules, constraints, and relationships in the database. This ensures that the database remains accurate and valid after the transaction is complete.
- **Example:** In a university database, if a student is enrolled in a course, the corresponding seats_available field should decrease. This ensures the database respects the integrity constraint.

3. Isolation

- **Definition:** In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system. • No transaction will affect the existence of any other transaction.
- **Explanation:** This prevents conflicts and ensures that transactions do not interfere with each other, even when they are executed simultaneously.
- **Example:** If two transactions are trying to update the same account balance, isolation ensures they are processed one after the other, not simultaneously, avoiding incorrect results.

4. Durability

- **Definition:** Durability guarantees that once a transaction is committed, its changes are permanent, even in the event of a system failure.
- **Explanation:** The system ensures that completed transactions are recorded in non-volatile storage, such as a database log, so that the changes can be recovered after a crash.
- **Example:** Once a user places an order on an e-commerce site, the order information is stored permanently, even if the system crashes afterward.

4. A transaction may be waiting for more time for an Exclusive (X) lock on an item, while a sequence of other transactions request and are granted as Shared (S) lock on the same item. What is this problem? How is it solved by two phase lock protocol?

The problem described is known as **starvation** or **indefinite waiting**. It occurs when a transaction keeps waiting for an **Exclusive (X) Lock** because a continuous sequence of **Shared (S) Locks** is granted to other transactions.

Understanding the Problem

1. Scenario:

- A transaction T1T1T1 requests an **Exclusive Lock (X-Lock)** on a data item AAA.
- Meanwhile, other transactions T2,T3,...,TnT2, T3, \ldots, TnT2,T3,...,Tn continuously request and are granted **Shared Locks (S-Locks)** on AAA.
- T1T1T1 remains in the queue, unable to acquire the lock, resulting in starvation.

2. Cause:

- **Shared Locks** are compatible with other **Shared Locks**, so multiple transactions can access the data item simultaneously in read mode.
- The **Exclusive Lock** requested by T1T1T1 cannot be granted until all current **Shared Locks** are released, but new **Shared Locks** keep coming in, preventing T1T1T1 from ever acquiring the **Exclusive Lock**.

How Two-Phase Locking (2PL) Solves This Problem

While the standard **Two-Phase Locking Protocol (2PL)** ensures serializability, it does not inherently prevent starvation. However, **practical implementations** of 2PL often include additional mechanisms to address this issue:

1. Priority-Based Locking

- Assign a **priority** to transactions based on factors such as their waiting time or timestamp.
- If a transaction T1T1T1 has been waiting for an **Exclusive Lock**, it is given higher priority, and no new **Shared Locks** are granted to other transactions until T1T1T1's lock is processed.
- This ensures fairness and prevents starvation.

2. Queue Management

- Lock requests are maintained in a **FIFO queue**.
- When a transaction requests an **Exclusive Lock**, it waits until all currently held locks are released, and no new **Shared Locks** are granted after T1T1T1 is in the queue.

3. Preemption or Timeout Mechanisms

- Some systems implement **timeouts**: if a transaction waits too long for a lock, the system aborts and restarts the transaction.

- This prevents indefinite waiting but may increase overhead due to transaction restarts.

5. What is conflict serializability? How to check schedule is conflict serializable schedule. Give one example.

Conflict serializability is a property of a schedule (sequence of operations) in a database system that ensures the schedule will produce the same result as some serial schedule (a schedule where transactions are executed one after another without interleaving).

A schedule is conflict serializable if it can be transformed into a serial schedule by swapping the order of non-conflicting operations. Two operations conflict if they meet the following conditions:

1. They belong to different transactions.
2. They access the same data item.
3. At least one of them is a write operation.

Checking Whether a Schedule is Conflict Serializable Or Not

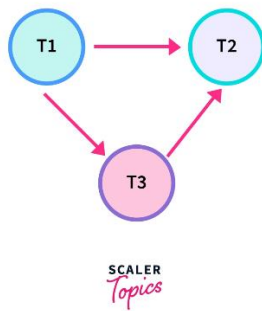
A non-serial schedule gets checked for conflict serializability using the following steps:

1. Figure out all the conflicting operations and enlist them.
2. Create a precedence graph. For every transaction in the schedule, draw a node in the precedence graph.
3. If $X_i(A)$ and $Y_j(A)$ represent a conflict pair, then draw an edge from T_i to T_j for each conflict pair. The precedence graph ensures that T_i gets executed before T_j .
4. The next step involves checking for any cycle formed in the graph. **A schedule is a conflict serializable if there is no cycle present.**

Example:

We have a schedule "S" having three transactions t_1 , t_2 , and t_3 working simultaneously. Let's form its precedence graph.

t1	t2	t3
R(x)		
	R(y)	
		R(y)
	W(y)	
W(x)		
		W(x)
	R(x)	
	W(x)	



So, order of its equivalent serial schedule is: $t1 \rightarrow t3 \rightarrow t2$

6. Explain the two phase lock protocol for concurrency control. Also explain its two versions : strict two phase lock protocol and rigorous two phase lock protocol.

Two-Phase Locking Protocol (2PL)

The Two-Phase Locking Protocol (2PL) is a concurrency control method used in database management systems to ensure that transactions are executed in a serializable manner, preventing anomalies such as lost updates, temporary inconsistencies, and uncommitted data. The 2PL protocol enforces a sequence of lock acquisition and release phases for transactions.

- **Shared Lock:** Data can only be read when a shared lock is applied. Data cannot be written. It is denoted as **lock-S**
- **Exclusive lock:** Data can be read as well as written when an exclusive lock is applied. It is denoted as **lock-X**

Two Phases of 2PL:

1. Growing Phase:

- In this phase, a transaction can acquire locks on data items but cannot release any locks.
- Once a transaction releases its first lock, it cannot acquire any new locks.

2. Shrinking Phase:

- In this phase, a transaction can release locks but cannot acquire any new locks.
- This phase starts as soon as the transaction releases its first lock.

By following these two phases, 2PL ensures that transactions maintain a consistent state and avoid conflicts, leading to serializable schedules.

Strict Two-Phase Locking Protocol

The Strict Two-Phase Locking (Strict 2PL) protocol is a variant of the standard 2PL that adds an additional constraint to enhance consistency and prevent cascading rollbacks.

Characteristics of Strict 2PL:

- In addition to the rules of 2PL, transactions under Strict 2PL hold all exclusive (write) locks until they commit or abort.
- This means that once a transaction acquires a lock, it cannot release it until the transaction is completed (commit or abort).

Advantages:

- **Avoids Cascading Rollbacks:** Since transactions do not release locks until they commit, other transactions cannot read uncommitted data, preventing cascading rollbacks.
- **Simplifies Recovery:** Ensures that once a transaction releases a lock, its effects are permanent, simplifying recovery processes.

Example: Consider two transactions, T1T1 and T2T2:

- T1T1: Acquires a write lock on item A, performs operations, then commits.
- T2T2: Tries to read item A while T1T1 is still running. Under Strict 2PL, T2T2 must wait until T1T1 commits, ensuring that T2T2 reads committed data.

Rigorous Two-Phase Locking Protocol

The Rigorous Two-Phase Locking (Rigorous 2PL) protocol is another variant of 2PL that imposes even stricter locking rules.

Characteristics of Rigorous 2PL:

- Similar to Strict 2PL, but extends the constraint to all locks (both read and write locks).
- Transactions hold all locks (read and write) until they commit or abort.

Advantages:

- **Maximum Consistency:** Ensures the highest level of consistency by holding all locks until transaction completion.
- **No Cascading Rollbacks:** Like Strict 2PL, it prevents cascading rollbacks since no locks are released until the transaction is fully completed.
- **Simplified Concurrency Control:** By holding all locks until the end, it simplifies the concurrency control mechanism.

Example: Consider two transactions, T1T1 and T2T2:

- T1T1: Acquires read and write locks on items A and B, performs operations, then commits.
- T2T2: Tries to read or write items A and B while T1T1 is still running. Under Rigorous 2PL, T2T2 must wait until T1T1 commits, ensuring that T2T2 only interacts with fully committed data.

7. Suppose a transaction T_i issues a read command on data item Q. How time-stamp based protocol decides whether to allow the operation to be executed or not using time-stamp based protocol of concurrency control. Explain the situations when each state transition occurs.

Timestamp-Based Protocol

Timestamp-Based Protocols are used in database management systems to ensure serializability by ordering transactions based on their timestamps. These protocols use timestamps to manage the order of operations and resolve conflicts. Each transaction is assigned a unique timestamp when it begins, which helps in deciding the order of execution of transactions.

Key Concepts

1. **Timestamp:** A unique identifier assigned to each transaction when it starts. Timestamps typically reflect the transaction's start time.
2. **Read Timestamp (RTS):** The latest timestamp at which a data item was read.
3. **Write Timestamp (WTS):** The latest timestamp at which a data item was written.

Basic Rules

Case 1 (Read) : Transaction T issues a **read(X) operation**

- i) If $TS(T) < WTS(X)$, then read(X) is rejected. T has to abort and be rejected.
- ii) If $WTS(X) \leq TS(T)$, then execute read(X) of T and update $RTS(X)$.

Case 2 (Write) : Transaction T issues a **write(X) operation**

- i) If $TS(T) < RTS(X)$ or if $TS(T) < WTS(X)$, then write is rejected
- ii) If $RTS(X) \leq TS(T)$ or $WTS(X) \leq TS(T)$, then execute write(X) of T and update $WTS(X)$.

8. Write a short note on :

i) Log based recovery

ii) Shadow Paging

Log-based recovery is a technique used in database management systems to ensure data integrity and consistency after a failure. This method relies on a log file, which records all transactions and their changes to the database. The log file is crucial for recovering the database to a consistent state.

Key Concepts

1. Transaction Log:

- A sequential record of all transactions and changes made to the database.
- Entries in the log include the transaction ID, type of operation (insert, update, delete), affected data items, and before-and-after values.

2. Write-Ahead Logging (WAL):

- Ensures that before any changes are made to the database, the corresponding log entries are written to disk.
- This ensures that in the event of a failure, all necessary information for recovery is available in the log.

Steps in Log-Based Recovery

1. Checkpointing:

- Periodically, the database system creates a checkpoint in the log.
- A checkpoint is a snapshot of the database state at a particular point in time.
- It helps to reduce the time required for recovery by limiting the amount of log data that needs to be processed.

2. Recovery Process:

- **Analysis Phase:** Determines the state of the transactions at the time of the failure by examining the log from the last checkpoint.
- **Redo Phase:** Reapplies all changes recorded in the log from the last checkpoint to ensure that all committed transactions are reflected in the database.
- **Undo Phase:** Reverses changes made by transactions that were active at the time of the failure but did not commit, ensuring that no partial transactions affect the database state.

9. What is recoverable schedule? Why is recoverability of schedule desirable? Are there any circumstance under Which it could be desirable to allow non recoverable scheduler? Explain your answer.

Recoverable Schedule

Definition: A recoverable schedule is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the commit operation of T_j .

For example : Consider following schedule, consider $A = 100$

T_1	T_2
R(A)	
$A = A + 50$	
W(A)	
	R(A)
	$A = A - 20$
	W(A)
	Commit
Some transaction...	
Commit	

Failure

- The above schedule is inconsistent if failure occurs after the commit of T_2 .
- It is because T_2 is dependable transaction on T_1 . A transaction is said to be dependable if it

contains a dirty read. • The dirty read is a situation in which one transaction reads the data immediately after the write operation of previous transaction.

• Now if the dependable transaction i.e. T2 is committed first and then failure occurs then if the transaction T1 makes any changes then those changes will not be known to the T2 . This leads to non recoverable state of the schedule. • To make the schedule recoverable we will apply the rule that – commit the independent transaction before any dependable transaction. • In above example independent transaction is T1 , hence we must commit it before the dependable transaction i.e. T2 .

Why Recoverability is Desirable:

Importance of Recoverability

1. **Consistency:** Prevents inconsistent states in the database by ensuring no transaction depends on uncommitted or aborted transactions.
2. **Avoid Cascading Abort:** If a non-recoverable schedule is allowed and Tj aborts, all dependent transactions (Ti) must also abort, leading to unnecessary work and inefficiency.
3. **Data Integrity:** Recoverable schedules ensure the durability and correctness of data in case of transaction failures..

Non-Recoverable Schedule and Its Issues

Non-Recoverable Schedule: A schedule is non-recoverable if a transaction commits before another transaction that has read data written by the first transaction. If the first transaction fails and needs to be rolled back, it could lead to inconsistencies since the second transaction has already committed.

Example:

1. T1: Writes value to item A.
2. T2: Reads item A written by T1.
3. T2: Commits.
4. T1: Fails and rolls back.

In this case, T2 commits before T1. If T1 fails and rolls back, T2 has already committed based on an invalid value, leading to inconsistency.

Are There Circumstances to Allow Non-Recoverable Schedules?

Non-recoverable schedules are generally undesirable but could be allowed in specific scenarios:

1. **High Performance with Read-Only Transactions:**

- If certain transactions are guaranteed to be **read-only** and do not write back changes, the impact of non-recoverability is minimal. However, this is rare and limited to specialized systems.

2. **Systems Without Strict Consistency Requirements:**

- In systems where **eventual consistency** suffices (e.g., some NoSQL databases or certain distributed systems), the cost of ensuring recoverability may outweigh its benefits. Non-recoverable schedules might be allowed temporarily, with corrective measures applied later.

3. **Optimistic Concurrency Control:**

- In certain **optimistic concurrency control** mechanisms, transactions proceed without locking, assuming conflicts are rare. Non-recoverable schedules might be allowed during execution but are validated at the commit phase, ensuring correctness.

4. **Performance Optimization:**

- If the system prioritizes **performance** over consistency (e.g., during analytics or speculative computations), non-recoverable schedules could be temporarily allowed with safeguards like periodic rollbacks or manual intervention.

10. To ensure atomicity despite failures we use Recovery Methods Explain in detail following Log-Based Recovery methods with example.

i) Deferred Database Modifications

ii) Immediate Database Modifications

To ensure **atomicity** in databases despite system failures (like crashes or power outages), **log-based recovery methods** are used. These methods maintain a **log** that records all changes made to the database. If a failure occurs, the log helps the system recover to a consistent state.

i) Deferred Database Modifications

In the **deferred database modification** approach:

- **All updates are recorded in the log**, but the actual updates to the database are **deferred** (delayed) until the transaction commits.

- No changes are written to the database until the transaction has reached its commit point.
- **Recovery:** Since changes are written only after a transaction commits, the recovery process applies only committed changes from the log to the database.

Steps:

1. Log the operation $\langle T_i, X, NewValue \rangle$
2. Perform the operation on the database only after T_i commits.

Advantages:

- Ensures atomicity easily because uncommitted changes are not written to the database.
- Simple recovery: Only committed transactions need to be redone during recovery.

Disadvantages:

- Higher latency, as updates are delayed until the transaction commits.

Example:

Transaction T_1 :

- T_1 : Read A , Update $A = A + 100$, Write A .

Log Entries:

1. $\langle T_1, Start \rangle$
2. $\langle T_1, A, NewValue = 200 \rangle$
3. $\langle T_1, Commit \rangle$

Recovery:

- Apply all log entries for committed transactions. Since T_1 committed, update $A = 200$ in the database.

ii) Immediate Database Modifications

In the **immediate database modification** approach:

- Updates are **immediately applied** to the database as soon as they are generated, even before the transaction commits.

- The log contains both the **old value** and the **new value** to allow undo and redo operations.
- **Recovery:** Requires undoing changes of uncommitted transactions and redoing changes of committed transactions.

Steps:

1. Log the operation $\langle T_i, X, \text{Old Value}, \text{New Value} \rangle$
2. Perform the operation on the database immediately.
3. Ensure the log entry is written to stable storage **before** the actual database update (Write-Ahead Logging - WAL).

Advantages:

- Reduces latency, as updates are applied immediately.
- Suitable for applications requiring immediate persistence of updates.

Disadvantages:

- Recovery is more complex because it involves both undoing and redoing changes.

Example:

Transaction T_2 :

- T_2 : Read A , Update $A = A + 100$, Write A .

Log Entries:

1. $\langle T_2, \text{Start} \rangle$
2. $\langle T_2, A, \text{OldValue} = 100, \text{NewValue} = 200 \rangle$
3. $\langle T_2, \text{Commit} \rangle$

Recovery:

1. If T_2 is committed: **Redo** T_2 's operations by applying $\text{NewValue} = 200$.
2. If T_2 is not committed: **Undo** T_2 's operations by reverting A to $\text{OldValue} = 100$.



UNIT 5

1. Explain how NOSQL databases are different than relational databases?

1. Data Modeling:

- **Relational Databases:** Use a structured data model with tables, rows, and columns. Each table represents an entity, and relationships are established using keys.
- **NoSQL Databases:** Can use various models including key-value, document, column-family, and graph. These models offer flexibility in representing complex data structures.

2. Schema:

- **Relational Databases:** Have a predefined schema. Changes to the schema can be complex and time-consuming.
- **NoSQL Databases:** Schema-less or have a flexible schema. This allows for easy adaptation and evolution of the data structure.

3. Query Language:

- **Relational Databases:** Use SQL (Structured Query Language) for defining and manipulating data. SQL is standardized and widely adopted.
- **NoSQL Databases:** Use various query languages depending on the type of database (e.g., MongoDB uses a JSON-like query language, Cassandra uses CQL). These languages are often designed to be intuitive for developers.

4. Scalability:

- **Relational Databases:** Typically scale vertically (adding more power to an existing machine).
- **NoSQL Databases:** Designed to scale horizontally (adding more machines). This makes them suitable for handling large amounts of data and high traffic.

5. Data Relationships:

- **Relational Databases:** Use foreign keys and JOIN operations to establish and manage relationships between tables.
- **NoSQL Databases:** Handle relationships differently depending on the model. Document and graph databases are particularly good at handling complex relationships without the need for JOINS.

6. Transaction Type:

- **Relational Databases:** Support ACID (Atomicity, Consistency, Isolation, Durability) transactions, ensuring reliable and predictable operations.
- **NoSQL Databases:** Many support BASE (Basically Available, Soft state, Eventually consistent) transactions, which provide high availability and scalability at the cost of immediate consistency. Some NoSQL databases also offer ACID compliance for specific use cases.

7. Performance:

- **Relational Databases:** Optimized for complex queries and transactions, but can become slower with large datasets and high traffic.
- **NoSQL Databases:** Optimized for high performance with large volumes of data and high throughput. The absence of JOINS and a flexible schema contributes to their speed.

8. Data Consistency:

- **Relational Databases:** Prioritize strong consistency due to ACID properties.
- **NoSQL Databases:** Often prioritize eventual consistency to achieve high availability and scalability. Some NoSQL databases offer tunable consistency levels.

9. Distributed Computing:

- **Relational Databases:** Typically not distributed, though some modern relational databases support distributed architectures.
- **NoSQL Databases:** Designed with distributed computing in mind, allowing them to efficiently manage large datasets across multiple nodes.

10. Fault Tolerance:

- **Relational Databases:** Rely on replication and backup strategies for fault tolerance.
- **NoSQL Databases:** Built to handle faults and partition tolerance natively. They often employ data replication across multiple nodes for reliability.

11. Data Partitioning:

- **Relational Databases:** Use sharding techniques for partitioning, which can be complex to implement and manage.

- **NoSQL Databases:** Naturally support partitioning and sharding, making it easier to distribute data across multiple nodes.

12. Data to Object Mapping:

- **Relational Databases:** Require an Object-Relational Mapping (ORM) layer to map database tables to object-oriented programming structures.
- **NoSQL Databases:** Often align more closely with object-oriented programming models, reducing the need for complex mappings.

2. Explain BASE properties with its significance. How soft state of system is depending on Eventual consistency property?

1. Basically Available:

- **Meaning:** The system guarantees availability, meaning that requests will be responded to even if some data isn't immediately available.
- **Significance:** This ensures that the system is always available for reads and writes, providing a degree of fault tolerance and high availability. It's crucial for systems that require 24/7 uptime and cannot afford to be offline.

1. Soft State:

- **Definition:** In a system with soft state, the state of the data can change over time, even without any external inputs. This is due to ongoing background activities such as replication, propagation, and synchronization of data across multiple nodes in a distributed system.
- **Behavior:** Soft state systems don't require immediate consistency. Instead, they allow for transient inconsistencies as data updates propagate throughout the system. This flexibility enables better performance and availability.

2. Eventual Consistency:

- **Definition:** Eventual consistency is a consistency model used in distributed computing that ensures that, given enough time without new updates, all replicas of the data will converge to the same state. The system guarantees that, eventually, all nodes will have the same data.
- **Behavior:** Eventual consistency allows for temporary inconsistencies but ensures that the system will become consistent over time. This is achieved by propagating updates asynchronously across all nodes.

Significance of BASE Properties:

BASE properties are significant in scenarios where high availability, scalability, and fault tolerance are more critical than immediate consistency. They are particularly well-suited for:

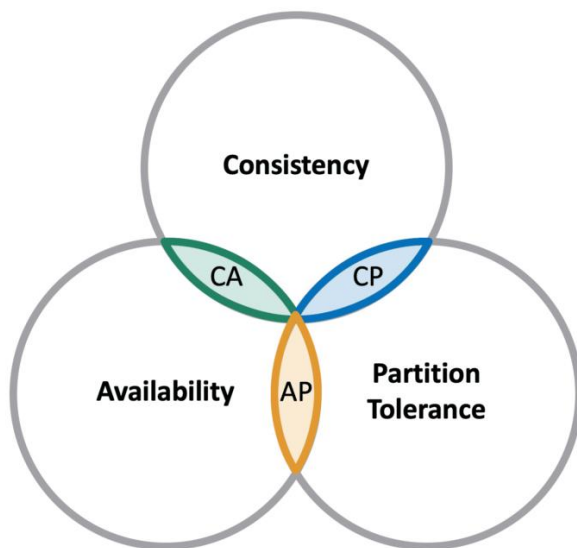
- **Large-scale Distributed Systems:** Where data is partitioned and replicated across many servers, such as in cloud-based applications.
- **High-throughput Systems:** Where the system needs to handle a large number of transactions per second, like social media platforms, e-commerce sites, and real-time analytics applications.
- **Fault-tolerant Systems:** Where the ability to recover from failures without significant downtime is essential.

Interdependence:

Soft State's Dependence on Eventual Consistency:

- The concept of soft state relies on eventual consistency because it assumes that data will converge to a consistent state over time. Without eventual consistency, the soft state model wouldn't work effectively, as there would be no guarantee that the data inconsistencies will resolve.
- Soft state systems allow for ongoing changes and data propagation, which aligns with the eventual consistency principle that all nodes will eventually reflect the same data after a period of time.

3. State and explain the concept of CAP theorem



The **CAP Theorem**, also known as Brewer's Theorem, is a fundamental principle in distributed systems that states it is impossible for a distributed data store to simultaneously provide all three of the following guarantees:

1. **Consistency (C)**
2. **Availability (A)**
3. **Partition Tolerance (P)**

Explanation of Each Component:

1. **Consistency:**

- **Definition:** Every read from the database returns the most recent write. This means that all nodes in a distributed system have the same data at any given time.
- **Significance:** Ensures that all users see the same data, providing a consistent view of the database.

2. **Availability:**

- **Definition:** Every request (read or write) receives a response, even if some of the nodes are down. The system is always available to handle requests.
- **Significance:** Ensures that the system remains operational and responsive, even during failures.

3. **Partition Tolerance:**

- **Definition:** The system continues to operate despite network partitions. A network partition occurs when there is a loss of communication between some nodes in a distributed system.
- **Significance:** Ensures that the system can handle network failures and continue to function correctly.

CAP Theorem States:

The theorem states that in the presence of a network partition (P), a distributed system can provide either consistency (C) or availability (A), but not both. Here's how it plays out:

- **Consistency and Partition Tolerance (CP):** The system will ensure that all nodes have the same data but may not be available for some operations during network partitions.

- **Availability and Partition Tolerance (AP):** The system will remain available for operations even during network partitions, but some data may be inconsistent.
- **Consistency and Availability (CA):** The system will provide a consistent and available service as long as there are no network partitions. However, this is impractical in distributed systems where partitions can happen.

Real-world Implications:

In designing distributed systems, architects often have to make trade-offs based on the CAP theorem:

- **CP Systems:** Examples include MongoDB, HBase, and other databases that prioritize consistency and partition tolerance, often sacrificing availability during network issues.
- **AP Systems:** Examples include Cassandra and DynamoDB, which prioritize availability and partition tolerance, allowing for some degree of data inconsistency.
- **CA Systems:** This combination is theoretically possible but impractical in distributed systems that need to operate across unreliable networks

4. Enlist the different types of NOSQL databases and explain with suitable examples.

Key Value Pair Based

Data is stored in key/value pairs. It is designed in such a way to handle lots of data and heavy load.

Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB(Binary Large Objects), string, etc.

For example, a key-value pair may contain a key like "Website" associated with a value like "Guru99".

Key	Value
Name	Joe Bloggs
Age	42
Occupation	Stunt Double
Height	175cm
Weight	77kg

It is one of the most basic NoSQL database example. This kind of NoSQL database is used as a collection, dictionaries, associative arrays, etc. Key value stores help the developer to store schema-less data. They work best for shopping cart contents.

Redis, Dynamo, Riak are some NoSQL examples of key-value store DataBases. They are all based on Amazon's Dynamo paper.

Column-based

Column-oriented databases work on columns and are based on Big Table paper by Google. Every column is treated separately. Values of single column databases are stored contiguously.

Row ID	Columns...		
1	Name	City	
	Ankita	Pune	
2	Name	City	email
	Kavita	Mumbai	kavita123@gmail.com

Column based NoSQL database

They deliver high performance on aggregation queries like SUM, COUNT, AVG, MIN etc. as the data is readily available in a column.

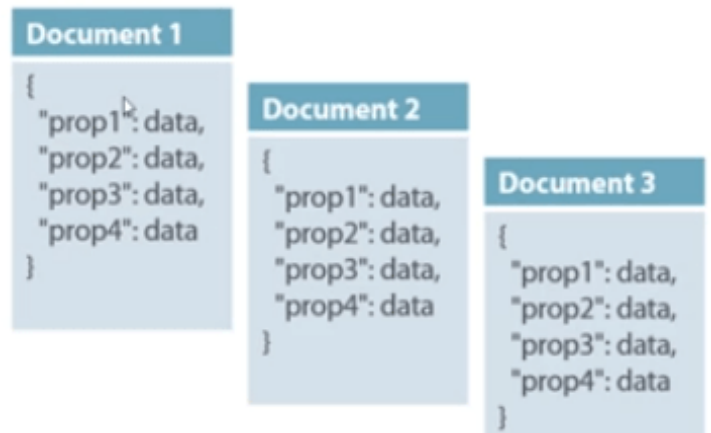
Column-based NoSQL databases are widely used to manage data warehouses, [business intelligence](#), CRM, Library card catalogs,

HBase, Cassandra, HBase, Hypertable are NoSQL query examples of column based database.

Document-Oriented

Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document. The document is stored in JSON or XML formats. The value is understood by the DB and can be queried.

Col1	Col2	Col3	Col4
Data	Data	Data	Data
Data	Data	Data	Data
Data	Data	Data	Data



Relational Vs. Document

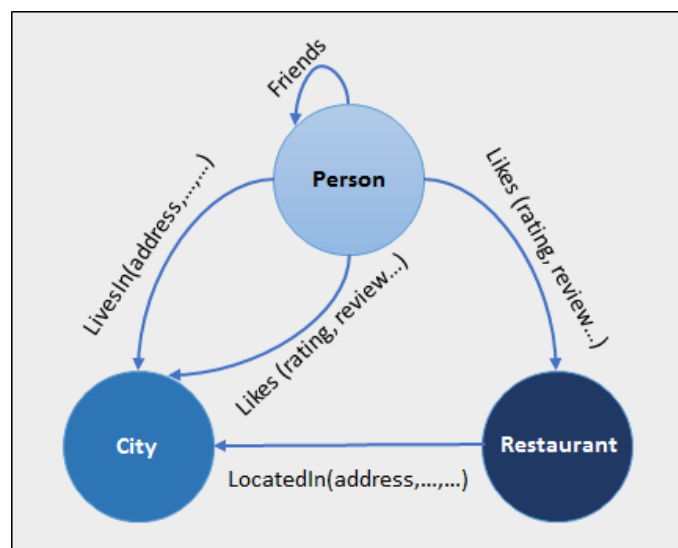
In this diagram on your left you can see we have rows and columns, and in the right, we have a document database which has a similar structure to JSON. Now for the relational database, you have to know what columns you have and so on. However, for a document database, you have data store like JSON object. You do not require to define which make it flexible.

The document type is mostly used for CMS systems, blogging platforms, real-time analytics & e-commerce applications. It should not use for complex transactions which require multiple operations or queries against varying aggregate structures.

Amazon SimpleDB, CouchDB, MongoDB, Riak, Lotus Notes, MongoDB, are popular Document originated [DBMS systems](#).

Graph-Based

A graph type database stores entities as well the relations amongst those entities. The entity is stored as a node with the relationship as edges. An edge gives a relationship between nodes. Every node and edge has a unique identifier.



Compared to a relational database where tables are loosely connected, a Graph database is a multi-relational in nature. Traversing relationship is fast as they are already captured into the DB, and there is no need to calculate them.

Graph base database mostly used for social networks, logistics, spatial data.

Neo4J, Infinite Graph, OrientDB, FlockDB are some popular graph-based databases.

5. Analyze the use of NOSQL databases in current social networking environment also explain need of NOSQL databases in social networking environment over RDBMS.

1. Characteristics of Social Networking Data

- **High Volume:** Social platforms generate massive amounts of data from posts, likes, shares, and user interactions.
- **Complex Relationships:** Social connections, such as friendships and followers, form highly interrelated and dynamic datasets.
- **Unstructured and Semi-Structured Data:** Content such as text, images, videos, and metadata lacks a fixed schema.
- **Real-Time Updates:** Social networks require low-latency data updates and real-time recommendations.
- **Global Reach:** Data needs to be distributed across geographies to ensure low latency and high availability.

2. How NoSQL Databases Address These Needs

a) Scalability

- **Horizontal Scalability:** NoSQL databases can scale out by adding more servers, unlike RDBMS, which typically require vertical scaling (expensive and limited).
- **Examples:** Facebook uses Cassandra for messaging, as it can handle massive scale across global data centers.

b) Handling Complex Relationships

- **Graph Databases** (e.g., Neo4j, Amazon Neptune): Ideal for managing and querying interconnected data like user relationships, follower graphs, and recommendations.
- **Example:** LinkedIn uses graph databases to power "People You May Know" recommendations.

c) Flexible Data Models

- **Document-Oriented Databases** (e.g., MongoDB): Store varied and evolving data structures (e.g., user profiles, posts with metadata).
- Example: Instagram uses Cassandra for managing user data and MongoDB for storing media content.

d) Real-Time Performance

- **Low Latency:** Key-value stores (e.g., Redis) are used for caching frequently accessed data like trending topics or notification counters.
- Example: Twitter uses Redis to handle real-time trends and search functionalities.

e) High Availability and Fault Tolerance

- NoSQL databases often include built-in replication and partitioning to ensure data is available even during failures.

3. Why NoSQL Over RDBMS for Social Networking

a) Schema Flexibility

- Social networks deal with rapidly evolving data requirements. NoSQL databases support schema-less or dynamic schemas, unlike RDBMS, which require predefined schemas.

b) Massive Scale and Performance

- RDBMS systems face bottlenecks when managing billions of rows across distributed servers. NoSQL databases, designed for distributed architectures, easily manage such scale.

c) Efficient Relationship Handling

- RDBMS require expensive JOIN operations to query relationships, leading to performance issues. Graph databases handle these relationships natively, providing faster queries for social connections.

d) Handling Unstructured Data

- Social platforms must store diverse content types (text, videos, images). NoSQL databases, particularly document-oriented ones, are well-suited for storing and querying such data.

e) Global Availability

- NoSQL databases like Cassandra and DynamoDB provide multi-region replication and data distribution out of the box, ensuring low latency and data availability across the globe.

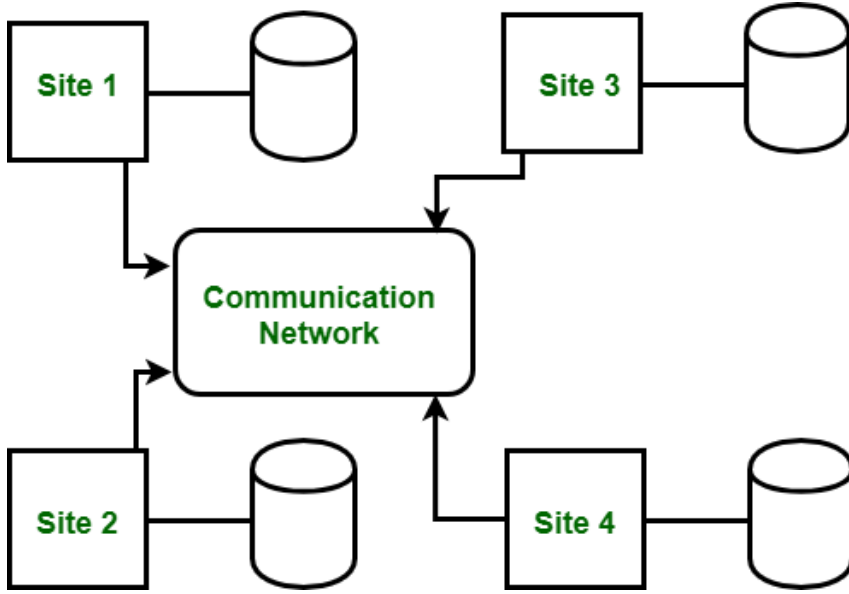
4. Example of NoSQL Adoption in Social Networking

- **Facebook:** Uses Cassandra for inbox search and real-time analytics.
- **Twitter:** Uses Redis for caching, Hadoop for analytics, and other NoSQL systems for real-time data processing.
- **LinkedIn:** Relies on graph databases for relationship recommendations and Neo4j for handling network connections.
- **Instagram:** Combines Cassandra and MongoDB for storing user-generated content and managing feeds.

6. Explain the difference between SQL and NOSQL database.

Features	SQL	NoSQL
Data Model	It is a relational data model. It represents data as tables with rows and columns.	It consists of key-value, document, column-family, and graph
Scalability	It is vertically scalable. We can increase the resources of a single server or machine to handle more load.	It is horizontally scalable. We have to add more machines or servers to distribute the workload.
Querying	The querying is done by using Structured Query Language (SQL).	Queries are API-based and often lack support for complex querying
Flexibility	SQL is less flexible for unstructured data	NoSQL is more flexible for unstructured or semi-structured data
Transactions	It supports transactions and is widely used	It has limited support and often requires custom implementation
Schema	SQL has a strict schema, pre-defined table structure	NoSQL has a flexible schema, dynamic and self-describing
Examples	MySQL, Oracle, Microsoft SQL Server	MongoDB, Cassandra, Redis, Neo4j

7. Draw and explain architecture of Distributed database system. State the reasons for building distributed database systems.



A Distributed Database System (DDBS) consists of multiple databases distributed across different physical locations. These databases are interconnected and communicate with each other to achieve common goals. The architecture of a DDBS typically includes the following components:

1. Local Databases:

- Databases that are stored at different physical locations, which could be on different servers or data centers.

2. Distributed DBMS (DDBMS):

- The software layer that manages the distributed databases, ensuring that they work together seamlessly. This layer handles data distribution, replication, and synchronization.

3. Communication Network:

- The network infrastructure that enables communication between the distributed databases. This can include local area networks (LANs), wide area networks (WANs), or the internet.

4. Data Distribution:

- The strategy used to distribute data across different locations. This can be done through:
 - **Horizontal Partitioning:** Dividing a table into subsets of rows.

- **Vertical Partitioning:** Dividing a table into subsets of columns.
- **Replication:** Creating copies of the same data in multiple locations.

5. **Global Catalog/Directory:**

- A centralized catalog that maintains metadata about the data distribution, schema, and structure of the distributed databases.

6. **Query Processor:**

- A component that optimizes and executes queries across the distributed databases. It translates high-level queries into efficient execution plans considering the distributed nature of the data.

7. **Transaction Manager:**

- Ensures that transactions are executed reliably across the distributed databases. It handles concurrency control, recovery, and maintaining ACID properties (Atomicity, Consistency, Isolation, Durability).

Reasons for Building Distributed Database Systems

1. **Scalability:**

- DDBS can handle increasing amounts of data and user loads by distributing the data across multiple locations. This allows for horizontal scaling, which involves adding more servers to manage the load.

2. **Reliability and Availability:**

- By replicating data across multiple locations, DDBS can ensure that data is available even if one or more sites fail. This redundancy increases the overall reliability and availability of the system.

3. **Performance:**

- By distributing the data closer to where it is needed, DDBS can reduce latency and improve response times. Queries can be executed locally on relevant subsets of data, reducing the need for long-distance data transfers.

4. **Geographical Distribution:**

- DDBS is ideal for organizations with operations in multiple geographical locations. It allows data to be stored and accessed locally, reducing the impact of network latency and improving performance for local users.

5. **Flexibility:**

- DDBS allows for heterogeneous databases, enabling different databases to coexist and work together within the same distributed system. This provides flexibility in choosing the best database technology for each specific need.

6. Load Balancing:

- By distributing the workload across multiple sites, DDBS can balance the load more effectively, preventing any single site from becoming a bottleneck.

7. Data Integration:

- DDBS allows for the integration of data from multiple sources, enabling a unified view of data across different locations and systems. This is essential for decision-making and analytics in large organizations.

8. Explain Structured, Semi-structured and Unstructured data types with examples.

Data Types: Structured, Semi-Structured, and Unstructured

Data can be categorized into three types based on its organization and how it can be processed. Here's an explanation of these data types with examples:

1. Structured Data

- **Description:**

Data that is highly organized and stored in a fixed format, such as rows and columns. It is easy to enter, search, and analyze using databases.

- **Key Features:**

- Follows a predefined schema.
- Easily searchable using query languages like SQL.
- Ideal for relational databases.

- **Examples:**

- **Spreadsheets:** Employee records in Excel (e.g., Name, Age, Salary, Department).
- **Relational Databases:** Customer data in MySQL, Oracle, or PostgreSQL.
- **Sensor Data:** Temperature or pressure readings stored in tabular formats.

- **Use Cases:**

- Financial transactions.
- Customer relationship management (CRM) systems.
- Inventory tracking.

2. Semi-Structured Data

- **Description:**

Data that does not conform to a fixed schema but still has some organizational structure through tags, markers, or keys. It is more flexible than structured data.

- **Key Features:**

- Schema-less or dynamically changing structure.
- Often stored in formats like JSON, XML, or YAML.
- Queryable using NoSQL databases.

- **Examples:**

- **JSON Data:** User profiles in MongoDB (e.g., {"name": "John", "age": 30, "interests": ["sports", "music"]}).
- **XML Files:** Configuration files in software applications.
- **Email:** Contains structured data (headers like sender and recipient) and unstructured data (message body).

- **Use Cases:**

- Web APIs for data exchange.
- Storing product catalogs with variable attributes.
- Log files in applications.

3. Unstructured Data

- **Description:**

Data that does not have a predefined format or structure. It is the most complex to store and analyze, often requiring advanced tools like AI and machine learning for processing.

- **Key Features:**

- No inherent structure.
- Stored in binary or raw formats like text, images, or videos.
- Requires specialized tools for indexing and analysis.

- **Examples:**

- **Text Documents:** Word files, PDFs.

- **Media Files:** Photos, videos, audio recordings.
- **Social Media Posts:** Tweets, Facebook posts, or Instagram captions.

- **Use Cases:**

- Image and video recognition in AI systems.
- Sentiment analysis from social media.
- Storage and retrieval of multimedia content.

Properties	Structured data	Semi-structured data	Unstructured data
Technology	It is based on Relational database table	It is based on XML/RDF(Resource Description Framework).	It is based on character and binary data
Transaction management	Matured transaction and various concurrency techniques	Transaction is adapted from DBMS not matured	No transaction management and no concurrency
Version management	Versioning over tuples,row,tables	Versioning over tuples or graph is possible	Versioned as a whole
Flexibility	It is schema dependent and less flexible	It is more flexible than structured data but less flexible than unstructured data	It is more flexible and there is absence of schema
Scalability	It is very difficult to scale DB schema	It's scaling is simpler than structured data	It is more scalable.
Robustness	Very robust	New technology, not very spread	—
Query performance	Structured query allow complex joining	Queries over anonymous nodes are possible	Only textual queries are possible

9. Describe the following operations with MongoDB syntax: i) Map-Reduce ii) Aggregation pipeline

An MongoDB, aggregation operations process multiple documents and return computed results. **Aggregation** in MongoDB is a powerful framework used to process and transform data. It allows you to perform complex queries, transformations, and computations on collections, such as filtering, grouping, sorting, and calculating derived results.

Aggregation Pipeline

The aggregation pipeline consists of a series of stages, each of which performs an operation on the input documents. The output of one stage becomes the input for the next stage. Common stages include:

- `$match`: Filters documents based on conditions (similar to SQL WHERE clause).
- `$group`: Groups documents by a specified field and applies aggregation operations (like SQL GROUP BY).
- `$project`: Reshapes the document fields (similar to SQL SELECT).
- `$sort`: Sorts documents based on a specified field.
- `$limit`: Limits the number of documents returned.
- `$skip`: Skips a specified number of documents.

Example: Aggregation Pipeline

Sample Data: Collection sales

```
json
[
  { "product": "A", "quantity": 10, "price": 50 },
  { "product": "B", "quantity": 5, "price": 30 },
  { "product": "A", "quantity": 15, "price": 50 },
  { "product": "C", "quantity": 7, "price": 40 }
]
```

Task 1: Total Sales Quantity for Each Product

javascript

Copy code

```
db.sales.aggregate([
  {
    $group: {
      _id: "$product",
      totalQuantity: { $sum: "$quantity" }
    }
  }
])
```

```
}  
]);
```

Output:

json

Copy code

```
[  
  { "_id": "A", "totalQuantity": 25 },  
  { "_id": "B", "totalQuantity": 5 },  
  { "_id": "C", "totalQuantity": 7 }  
]
```

i) Map-Reduce in MongoDB

Map-Reduce is a data processing technique in MongoDB used for performing aggregation and data transformation on large datasets. It processes data in two stages:

Map (filters and processes data) and **Reduce** (combines intermediate results).

While MongoDB now recommends the **aggregation pipeline** for most use cases, Map-Reduce is still useful for certain complex operations that cannot be handled by the aggregation framework.

Syntax for Map-Reduce

javascript

```
db.collection.mapReduce(  
  mapFunction,    // The mapping function  
  reduceFunction, // The reducing function  
  {  
    out: { inline: 1 }, // Output options (e.g., inline or a collection)  
    query: {},          // Query to filter documents (optional)  
    sort: {},           // Sort criteria (optional)  
    limit: 1000         // Limit on documents processed (optional)  
  })
```

- 1) map Function : It uses emit() function in which it takes two parameters key and value key. Here the key is on which we make groups (such as group by name, or age) and the

second parameter is on which aggregation is performed like avg(), sum() is calculated on.

- 2) reduce Function : This is a function in which we perform aggregate functions like avg(), sum()
- 3) out : It will specify the collection name where the result will be stored.
- 4) query : We will pass the query to filter the result set.
- 5) sort : It specifies the optional sort criteria.
- 6) limit : It specifies the optional maximum number of documents to be returned.

10.Explain the CRUD operations used in MongoDB with example.

Certainly! CRUD operations (Create, Read, Update, Delete) are the four basic operations for interacting with a database. Let's explore how these operations work in MongoDB with examples using a hypothetical collection named users.

1. Create (Insert)

The insertOne or insertMany method is used to create new documents in a collection.

Example:

javascript

```
// Insert a single document
db.users.insertOne({
  name: "Alice",
  age: 25,
  email: "alice@example.com"
});
```

```
// Insert multiple documents
db.users.insertMany([
  { name: "Bob", age: 30, email: "bob@example.com" },
  { name: "Charlie", age: 35, email: "charlie@example.com" }
]);
```

2. Read (Query)

The find method is used to read or query documents from a collection. You can specify criteria to filter the results.

Example:

javascript

```
// Find all documents
```

```
db.users.find({});
```

```
// Find a document with a specific criterion
```

```
db.users.find({ name: "Alice" });
```

```
// Find documents with an age greater than 30
```

```
db.users.find({ age: { $gt: 30 } });
```

3. Update

The `updateOne`, `updateMany`, or `replaceOne` methods are used to update existing documents.

Example:

```
javascript
```

```
// Update a single document
```

```
db.users.updateOne(  
  { name: "Alice" }, // Filter  
  { $set: { age: 26 } } // Update operation  
);
```

```
// Update multiple documents
```

```
db.users.updateMany(  
  { age: { $lt: 30 } }, // Filter  
  { $set: { status: "young" } } // Update operation  
);
```

```
// Replace an entire document
```

```
db.users.replaceOne(  
  { name: "Alice" }, // Filter  
  { name: "Alice", age: 26, email: "alice_new@example.com" } // New document  
);
```

4. Delete

The `deleteOne` or `deleteMany` methods are used to remove documents from a collection.

Example:

```
javascript
```

```
// Delete a single document
```

```
db.users.deleteOne({ name: "Alice" });
```

```
// Delete multiple documents
db.users.deleteMany({ age: { $gt: 30 } }
```

UNIT 6

1. Difference between relational databases and object relational databases with example

OODBMS	DBMS
OODBMS stands for Object Oriented Database Management System that supports creating and modelling of data as objects.	DBMS is any Database Management System. The most popular DBMS are relational database management systems in which we store everything as a relation between entities.
Handles larger and complex data than RDBMS.	Handles comparatively simpler data.
Data Handling - Stores data as well as methods to use it.	Data Handling - RDBMS stores only data.
OODBMS is object-oriented.	RDBMS is table-oriented.
OODBMS uses inheritance and encapsulation to reduce data redundancy.	Normalization is used to eliminate data redundancy in RDBMS.
Examples - Object Database, Objectivity/DB, ObjectStore, Cache, and, ZODB	Examples - MSSQL, MySQL, and Oracle

2. Describe the significance of JSON data type and object. Discuss with syntax all JSON data types with suitable example.

Significance of JSON Data Type and Object

JSON (JavaScript Object Notation) is a lightweight data interchange format that is easy for humans to read and write and easy for machines to parse and generate. It is widely used for transmitting data between a server and a web application, as well as for storing data in a structured manner.

Significance:

1. **Human-Readable:** JSON's text format is easy for humans to read and understand, making it ideal for configuration files and data interchange.

2. **Language Independent:** JSON is language agnostic, meaning it can be used with virtually any programming language, making it highly versatile.
3. **Interoperability:** It allows for easy data exchange between different systems and applications, enhancing interoperability.
4. **Lightweight:** JSON's lightweight nature makes it efficient for data transmission over networks, contributing to faster performance.
5. **Hierarchical Structure:** JSON's ability to represent complex data structures, such as arrays and objects, makes it suitable for a wide range of use cases.

JSON Data Types

JSON supports the following data types:

1. **Object:**

- An unordered collection of key/value pairs. Each key/value pair is separated by a comma.
- **Syntax:**

```
json
{
  "name": "Alice",
  "age": 25,
  "email": "alice@example.com"
}
```

2. **Array:**

- An ordered list of values. Arrays can contain elements of different data types.
- **Syntax:**

```
json
[
  "apple",
  "banana",
  "cherry"
]
```

]

3. **String:**

- A sequence of characters enclosed in double quotes. Strings can contain Unicode characters.
- **Syntax:**

json

```
{  
  "message": "Hello, world!"  
}
```

4. **Number:**

- Represents both integer and floating-point numbers. JSON does not distinguish between different types of numbers.
- **Syntax:**

json

```
{  
  "integer": 42,  
  "float": 3.14  
}
```

5. **Boolean:**

- Represents a logical value: true or false.
- **Syntax:**

json

```
{  
  "isActive": true,  
  "isVerified": false  
}
```

6. **Null:**

- Represents an empty or non-existent value.
- **Syntax:**

```
json  
{  
  "middleName": null  
}
```

3. What is object relational database? What are its advantages and disadvantages?

Object-Relational Database (ORD)

An **Object-Relational Database (ORD)**, or Object-Relational Database Management System (ORDBMS), is a type of database management system that combines the features of both relational databases and object-oriented databases. ORDBMS aims to provide the benefits of object-oriented databases while maintaining the robustness and simplicity of relational databases. It extends the relational database system with object-oriented features, such as support for complex data types, inheritance, and polymorphism.

Features of Object-Relational Databases:

1. Complex Data Types:

- ORDBMS supports complex data types beyond the standard relational types (integers, strings, etc.). This includes user-defined types, composite types, and arrays.

2. Inheritance:

- Objects in ORDBMS can inherit properties and methods from other objects, similar to inheritance in object-oriented programming.

3. Encapsulation:

- Data and methods (functions) that operate on the data are encapsulated within objects, allowing for better data integrity and abstraction.

4. Polymorphism:

- ORDBMS supports polymorphism, which means the same operation can behave differently on different classes of objects.

5. **Support for SQL:**

- ORDBMS extends SQL to include object-oriented constructs, enabling more complex and efficient queries.

Advantages of Object-Relational Databases:

1. **Enhanced Modeling Capabilities:**

- ORDBMS can model complex data structures and relationships more naturally than traditional relational databases. This is particularly useful for applications that deal with complex data, such as multimedia, geographic information systems (GIS), and engineering design.

2. **Reusability:**

- The use of object-oriented concepts allows for reusability of objects and code, reducing development time and effort.

3. **Data Integrity and Consistency:**

- By encapsulating data and behavior within objects, ORDBMS can enforce data integrity and consistency more effectively.

4. **Scalability:**

- ORDBMS can handle large volumes of complex data more efficiently, making it suitable for enterprise-level applications.

5. **Compatibility with Relational Databases:**

- ORDBMS maintains compatibility with relational databases, allowing for easier migration and integration of existing relational data.

Disadvantages of Object-Relational Databases:

1. **Complexity:**

- ORDBMS can be more complex to design and manage compared to traditional relational databases. The added features and capabilities require a deeper understanding and more sophisticated design.

2. **Performance Overhead:**

- The additional object-oriented features can introduce performance overhead, especially for applications that do not require such complexity.

3. Limited Support and Adoption:

- While some ORDBMS systems are widely used, the overall adoption is limited compared to traditional relational databases. This can lead to fewer resources, tools, and community support.

4. Steeper Learning Curve:

- Developers and database administrators may face a steeper learning curve when working with ORDBMS, as it combines concepts from both relational and object-oriented paradigms.

Example of ORDBMS:

PostgreSQL is a widely-used ORDBMS that supports object-oriented features such as user-defined types, inheritance, and polymorphism.

4. Write a short note on emerging databases : i) Active Databases ii) Main Memory Databases

i) Active Databases

Description: Active databases are a type of database system that incorporate event-driven architecture to automate certain tasks. They use rules called triggers and events to monitor and respond to changes or specific conditions within the database. This makes it possible to automate responses to data modifications without manual intervention.

Features:

- **Triggers:** Special procedures that are automatically executed in response to specific events on a particular table or view.
- **Events:** Defined occurrences, such as data modifications (inserts, updates, deletes) or time-based events, that activate triggers.
- **Rules:** Conditions and actions defined to specify what should happen when an event occurs.

Example Use Case:

- **Monitoring:** In a banking system, an active database can automatically monitor account balances and trigger alerts or transactions if a balance drops below a certain threshold.
- **Auditing:** Automatically logging changes to sensitive data for auditing purposes.

- **Business Processes:** Automating business workflows, such as updating inventory levels when new sales orders are placed.

ii) Main Memory Databases

Description: Main memory databases (MMDB), also known as in-memory databases, store data primarily in the main memory (RAM) rather than on disk. This allows for extremely fast data access and processing since accessing data in memory is much quicker than disk access.

Features:

- **High Performance:** Data is accessed and manipulated at very high speeds due to its presence in RAM.
- **Low Latency:** Minimal delay in data retrieval and transaction processing.
- **Durability:** Uses techniques such as periodic snapshots, transaction logs, and replication to ensure data durability despite being stored in volatile memory.
- **Scalability:** Can handle large datasets by utilizing distributed architectures and scaling horizontally.

Example Use Case:

- **Real-time Analytics:** Applications that require real-time data analysis and rapid response times, such as financial trading systems, where the speed of data processing is critical.
- **Caching:** Acting as a caching layer to store frequently accessed data for applications with high read and write operations, reducing latency and improving performance.
- **Gaming:** Supporting real-time gaming environments where quick data retrieval and updates are crucial for seamless gameplay experiences.

5. Write a short note on complex data types : i) Semi-structured data ii)

Features of semi-structured data models

i) Semi-structured Data:

Definition: Semi-structured data is a type of data that is not purely structured, but also not completely unstructured. It contains some level of organization or structure, but does not conform to a rigid schema or data model, and may contain elements that are not easily categorized or classified.

Semi-structured data is typically characterized by the use of metadata or tags that provide additional information about the data elements.

Examples:

- **JSON (JavaScript Object Notation):** Widely used in web applications for data exchange.
- **XML (eXtensible Markup Language):** Used for representing structured data in a text format.
- **YAML (YAML Ain't Markup Language):** A human-readable data serialization standard often used for configuration files.

Use Cases:

- **Web Data Exchange:** JSON and XML are extensively used for data interchange between web servers and clients.
- **Configuration Management:** YAML and XML are used to define configurations in software applications.
- **NoSQL Databases:** Many NoSQL databases like MongoDB and CouchDB store data in semi-structured formats.

ii) Features of Semi-structured Data Models:

1. Flexibility:

- **Schema-less or Flexible Schema:** Semi-structured data models do not require a predefined schema, allowing for variations in data structure. This flexibility makes it easy to adapt to changes in data requirements.

2. Hierarchical Structure:

- **Nested Elements:** Semi-structured data can contain nested elements, enabling the representation of complex data relationships within a single document.

3. Self-Describing Nature:

- **Metadata Included:** The data contains metadata (tags or markers) that describe the data elements, making it self-descriptive. This feature simplifies data parsing and interpretation.

4. Interoperability:

- **Data Exchange:** Semi-structured data formats like JSON and XML are platform-independent and widely accepted, facilitating easy data exchange between different systems and applications.

5. Ease of Use:

- **Human-Readable:** Formats like JSON, XML, and YAML are human-readable, making it easier for developers to write, read, and debug data.

6. Storage and Retrieval:

- **Efficient Storage:** Semi-structured data can be efficiently stored in NoSQL databases, which are designed to handle flexible data models.
- **Query Capability:** Many NoSQL databases provide powerful query capabilities to extract and manipulate semi-structured data.

6. Explain how encoding and decoding of JSON object is done JAVA with example.

Encoding and Decoding JSON Objects

JSON (JavaScript Object Notation) is a lightweight data-interchange format widely used for data transfer between servers and clients. Encoding (serialization) and decoding (deserialization) are the processes of converting data to and from JSON format.

Encoding JSON (Serialization)

- **Definition:** Converting a Python, JavaScript, or other language object (like a dictionary, list, etc.) into a JSON string.
- **Purpose:** To transmit data over a network or save it in a JSON file.

Examples

1. In JavaScript:

```
javascript
```

```
const obj = { name: "Alice", age: 25, city: "New York" };
```

```
const jsonString = JSON.stringify(obj); // Encoding to JSON string
```

```
console.log(jsonString);
```

```
// Output: {"name":"Alice","age":25,"city":"New York"}
```

Decoding JSON (Deserialization)

- **Definition:** Converting a JSON string back into an object (e.g., dictionary in Python, object in JavaScript).
- **Purpose:** To reconstruct the original data structure for use in a program.

Examples

1. In JavaScript:

javascript

Copy code

```
const jsonString = '{"name":"Alice","age":25,"city":"New York"}';  
const obj = JSON.parse(jsonString); // Decoding to object  
console.log(obj.name); // Output: Alice
```

7. What is Deductive Database. Explain its features and state its advantages over traditional database.

Deductive Database:

A **deductive database** is a type of database system that provides advanced capabilities for querying and reasoning about the stored data. It combines traditional database management systems (DBMS) with deductive logic programming, enabling the derivation of new information from existing data through the use of rules and facts.

Features of Deductive Database:

1. Rule-Based Queries:

- Deductive databases use rules to derive new information from existing data. These rules are typically written in a logic programming language such as Datalog.

2. Inference Mechanism:

- The system can infer new facts from existing data using a set of rules. This is achieved through a built-in inference engine that processes the rules to generate new data.

3. Recursive Queries:

- Deductive databases support recursive queries, allowing for complex hierarchical and transitive relationships to be queried and processed efficiently.

4. Declarative Query Language:

- They use a declarative query language, where users specify what they want to retrieve rather than how to retrieve it. This simplifies the querying process and makes it more intuitive.

5. Integration of Facts and Rules:

- Facts (data) and rules (logic) are integrated, enabling seamless querying and reasoning. The database stores both data and the logic needed to derive additional information.

Advantages Over Traditional Databases:

1. Complex Query Handling:

- Deductive databases can handle complex queries involving recursion and transitive relationships more efficiently than traditional relational databases.

2. Automated Inference:

- The ability to automatically infer new information from existing data through rules provides powerful reasoning capabilities that are not available in traditional databases.

3. Enhanced Data Modeling:

- They provide richer data modeling capabilities by incorporating logical rules, which can represent complex relationships and constraints within the data.

4. Reduced Redundancy:

- By deriving information through rules rather than storing it explicitly, deductive databases can reduce data redundancy and storage requirements.

5. Improved Query Flexibility:

- The use of a declarative query language allows for more flexible and intuitive query formulation, making it easier for users to express complex queries.

Example:

Consider a simple deductive database for a family tree:

Facts:

plaintext

parent(john, mary).

parent(mary, susan).

parent(mary, tom).

Rules:

plaintext

ancestor(X, Y) :- parent(X, Y).

ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).

Query:

plaintext

?- ancestor(john, susan).

Explanation:

- The facts define the parent-child relationships.
- The rules define the ancestor relationship using recursive logic.
- The query asks whether John is an ancestor of Susan, which the deductive database can infer based on the given facts and rules.

Conclusion:

Deductive databases offer advanced querying and reasoning capabilities through the use of logical rules and inference mechanisms. They provide significant advantages over traditional databases in handling complex queries, automating inference, and enhancing data modeling, making them suitable for applications requiring sophisticated data analysis and reasoning.

8. What are spatial data. Explain Geographic and Geometric data.

Spatial Data

Spatial data, also known as geospatial data, refers to information about the physical location and shape of objects on the Earth's surface. This data is used to describe the location, shape, and relationship between geographic features such as points, lines, and polygons. Spatial data can be represented in various forms, including maps, geographic coordinates, and vector or raster formats.

Geographic Data

Geographic data (or geospatial data) specifically pertains to information about objects and phenomena that are located on the Earth's surface. This data is often used in

Geographic Information Systems (GIS) to analyze and visualize the spatial relationships and patterns in the real world. Geographic data typically includes the following components:

1. Location Information:

- Coordinates (latitude and longitude) that define the position of a feature on the Earth's surface.
- Example: The coordinates of a landmark, such as the Eiffel Tower (48.8584° N, 2.2945° E).

2. Attribute Information:

- Descriptive data that provides additional information about the geographic feature.
- Example: Attributes of a river, such as its name, length, and flow rate.

3. Spatial Relationships:

- Information about how geographic features are related to each other spatially, such as adjacency, proximity, or containment.
- Example: The relationship between a city and its surrounding suburbs.

Geometric Data

Geometric data refers to the mathematical representation of shapes and structures in a coordinate system. Unlike geographic data, which is tied to the Earth's surface, geometric data can describe any shape in any coordinate system, including abstract or non-geographic contexts. Geometric data includes:

1. Points:

- A single location in a coordinate system defined by a set of coordinates.
- Example: A point (2, 3) in a 2D Cartesian coordinate system.

2. Lines:

- A series of connected points that form a straight or curved path.
- Example: A line segment connecting points (2, 3) and (5, 7).

3. Polygons:

- A closed shape defined by a series of connected line segments.
- Example: A triangle with vertices at points (1, 1), (4, 1), and (2.5, 4).

9. What is the significance of XML databases? Explain with proper example when to use XML database.

- XML stands for eXtensible Markup Language.
- This scripting language is similar to HTML. That means, this scripting language contains various tags. But these tags are not predefined tags, in-fact user can define his own tags.

- Thus HTML is designed for representation of data on the web page whereas the XML is designed for transport or to store data.

XML databases are specialized systems designed to store, retrieve, and manage XML data efficiently. XML (eXtensible Markup Language) is a widely used format for representing semi-structured data in a hierarchical structure. XML databases can either be native (designed to store XML directly) or relational databases enhanced with XML capabilities.

When to Use XML Databases

1. Hierarchical Data:

- When the data naturally fits a tree-like structure.
- Example: Storing a **bookstore catalog**, where each book has nested attributes like title, author, publisher, and price.

2. Variable Schema:

- When the data schema may frequently change or varies between records.
- Example: Managing **metadata for digital assets**, where each asset type (e.g., image, video) might have different metadata.

3. Data Exchange:

- When data needs to be shared or exchanged between systems in XML format.
- Example: A system that imports and exports data in XML for integration with other applications or APIs.

4. Document Storage:

- When storing large amounts of textual data or semi-structured documents.
- Example: Archiving **legal contracts** or **technical manuals**.

5. Content Management Systems:

- For managing dynamic web content where structure varies across different pages.
- Example: Storing **blog posts** with varying content structures.

Example: Using XML Database

Consider a **bookstore application** that stores books with varying attributes:

XML Structure:

```

<bookstore>

  <book>

    <title>Effective Java</title>

    <author>Joshua Bloch</author>

    <price>45.00</price>

  </book>

  <book>

    <title>XML in a Nutshell</title>

    <author>Elliottte Harold</author>

    <publisher>O'Reilly</publisher>

    <price>39.95</price>

  </book>

</bookstore>

```

In this example:

- Each <book> element can have different attributes, and an XML database can store this data efficiently.
- Queries like "Find all books cheaper than \$50" can be handled using **XQuery**:

xquery

for \$book in /bookstore/book

where \$book/price < 50

return \$book/title

10.Explain Table inheritance with example.

Table Inheritance

Table inheritance is a concept in relational databases where a table (known as a child table) inherits columns and constraints from one or more other tables (known as parent tables). This approach is used to model hierarchical data and relationships between entities in a more structured and organized manner.

Types of Table Inheritance:

1. **Single Table Inheritance:**

- All classes are represented in a single table. A type discriminator column is used to distinguish between different classes.

2. **Class Table Inheritance:**

- Each class is represented in its own table. The child table includes a foreign key reference to the parent table.

3. **Concrete Table Inheritance:**

- Each class is represented by its own table with no shared tables for parent classes. Columns are duplicated in each child table.

UNIT 3

- 1. What is normalization ? What is the need of normalized database ? Explain 1NF, 2NF, 3NF and BCNF with example.**
- 2. Explain with example requirements of first normal form.**
- 3. Explain with example requirements of second normal form.**
- 4. Explain 3NF and BCNF and give its example. Also enlist their differences.**
- 5. Elaborate the significance of codd's rule. Explain 12 rules proposed by codd's.**
- 6. What are the desirable properties of decomposition? Explain it with example.**
- 7. Explain partial and full dependencies and transitive dependencies with example.**
- 8. What is the impact of insert, update & delete anomaly on overall design of database? How normalization is used to remove these anomalies?**
- 9. What is functional dependency? Explain its use in database design. Consider the following schema. Student (RollNo, Branch_code, Marks_Obtained, Exam_Name, Total_Marks)
Identify the functional dependencies and check whether the given schema is in 3NF or not. If not justify and convert the schema into 3NF.**
- 10. What is decomposition? Consider the relation F (FN, PN, C, D) with the following Functional Dependencies:
FD1: FN, PN $\rightarrow$ C
FD2: C $\rightarrow$ D
FD3: D $\rightarrow$ F
If F is decomposed in to F1 (FN,PN,C) and F2 (C,D). check decomposition is lossless or lossy?**
- 11. What are relational integrity constraints. Explain with example Domain constraints, Referential-Integrity and enterprise constraints.**

UNIT 4

- 1. Transaction during its execution should be in one of the different states at any point of time, explain the different states of transactions during its execution.**
- 2. Explain the concept of conflict serializability with suitable example. Since every conflict-serializable schedule is view serializable, why do we emphasize conflict serializability rather than view serializability?**
- 3. State and explain the ACID properties.**
- 4. A transaction may be waiting for more time for an Exclusive (X) lock on an item, while a sequence of other transactions request and are granted as Shared (S) lock on the same item. What is this problem? How is it solved by two phase lock protocol?**
- 5. What is conflict serializability? How to check schedule is conflict serializable schedule. Give one example.**
- 6. Explain the two phase lock protocol for concurrency control. Also explain its two versions : strict two phase lock protocol and rigorous two phase lock protocol.**
- 7. Suppose a transaction T_i issues a read command on data item Q. How time-stamp based protocol decides whether to allow the operation to be executed or not using time-stamp based protocol of concurrency control. Explain the situations when each state transition occurs.**
- 8. Write a short note on :**
 - i) Log based recovery**
 - ii) Shadow Paging**
- 9. What is recoverable schedule? Why is recoverability of schedule desirable? Are there any circumstance under Which it could be desirable to allow non recoverable scheduler? Explain your answer.**

UNIT 5

- 1. Explain how NOSQL databases are different than relational databases?**
- 2. Explain BASE properties with its significance. How soft state of system is depending on Eventual consistency property?**
- 3. State and explain the concept of CAP theorem**
- 4. Enlist the different types of NOSQL databases and explain with suitable examples.**
- 5. Analyze the use of NOSQL databases in current social networking environment also explain need of NOSQL databases in social networking environment over RDBMS.**
- 6. Explain the difference between SQL and NOSQL database.**
- 7. Draw and explain architecture of Distributed database system. State the reasons for building distributed database systems.**
- 8. Explain Structured, Semi-structured and Unstructured data types with examples.**
- 9. Describe the following operations with MongoDB syntax: i) Map-Reduce ii) ii)Aggregation pipeline**
- 10.Explain the CRUD operations used in MongoDB with example.**

UNIT 6

- 1. Difference between relational databases and object relational databases with example**
- 2. Describe the significance of JSON data type and object. Discuss with syntax all JSON data types with suitable example.**
- 3. What is object relational database? What are its advantages and disadvantages?**
- 4. Write a short note on emerging databases : i) Active Databases ii) Main Memory Databases**
- 5. Write a short note on complex data types : i) Semi-structured data ii) Features of semi-structured data models**
- 6. Explain how encoding and decoding of JSON object is done JAVA with example.**
- 7. What is Deductive Database. Explain its features and state its advantages over traditional database.**
- 8. What are spatial data. Explain Geographic and Geometric data.**
- 9. What is the significance of XML databases? Explain with proper example when to use XML database.**
- 10. Explain Table inheritance with example.**